Dialogue Injection Attack: Jailbreaking LLMs Through Context Manipulation

Wenlong Meng, Fan Zhang, Wendao Yao, Zhenyuan Guo, Yuwei Li, Chengkun Wei, and Wenzhi Chen, *Member, IEEE*

***Abstract*—Large language models (LLMs) have demonstrated significant utility in a wide range of applications; however, their deployment is plagued by security vulnerabilities, notably jailbreak attacks. These attacks manipulate LLMs to generate harmful or unethical content by crafting adversarial prompts. While much of the current research on jailbreak attacks has focused on single-turn interactions, it has largely overlooked the impact of historical dialogues on model behavior. Although recent studies have explored multi-turn jailbreak attacks, they generally assume that the attacker can only manipulate the user prompt. In contrast, we highlight that an attacker can also control the model's previous outputs. To this end, we introduce DIA, a new paradigm that leverages fabricated dialogue history to enhance jailbreak effectiveness. DIA operates in a black-box setting, requiring only access to the chat API or knowledge of the LLM's chat template. We propose two methods for constructing adversarial historical dialogues: one adapts gray-box prefilling attacks, and the other exploits deferred responses. Our experiments demonstrate that DIA achieves state-of-the-art attack success rates on recent LLMs, including Llama-3.1 and GPT-4o. Additionally, we show that DIA can bypass 6 different defense mechanisms, highlighting its robustness. Code is available at https://github.com/meng-wenlong/DIA**

***Index Terms*—Jailbreak attacks, dialogue injection, safety alignment.**

I. Introduction

THE advent of *large language models* (LLMs) such as GPT-4 [1] and PaLM [2] has revolutionized the field of natural language processing, exhibiting great power in natural language generation. Due to their robust generative abilities, LLMs have been deployed across a multitude of content generation applications, including code generators [3], translators [4], and personalized recommendation systems [5]. Among these, chatbots represent the most popular application, exemplified by OpenAI ChatGPT and Google Bard. Since LLMs have been trained on broad web corpora that include both compliant and non-compliant content, they may generate harmful content when prompted [6], [7]. To mitigate these risks, LLM developers employ alignment techniques such as *reinforcement learning from human feedback* (RLHF) [8] to ensure outputs conform to human ethical standards. This alignment aims to ensure that LLMs like ChatGPT adhere to ethical norms and foster constructive engagement.

However, the security mechanisms integrated into LLMs are not foolproof. Security and privacy concerns have been widely studied in AI-driven systems, including graph neural networks [9], semantic communications [10], and private information retrieval [11]. LLMs face *jailbreak attacks*, where adversaries craft specialized prompts to coax LLMs into producing harmful or unethical outputs [12], [13]. These attacks employ various techniques such as hypnosis [14], sensitive words disguise [15], and tense shift [16]. Such manipulated outputs could facilitate further malicious activities, including the extraction of personal information from training data [17] or the creation of malware [18]. Studying these jailbreak attacks is crucial for developers to enhance the security of LLM applications and improve alignment with safety standards.

Current research on jailbreak methodologies predominantly focuses on single-turn interactions, where all adversarial content is compressed into a single prompt. However, the influence of dialogue history plays a significant role in shaping the outputs of LLMs, especially in chatbot applications. Existing LLM chatbots utilize a *chat template* to concatenate historical dialogues with the current prompt before submitting it to the backend LLM for continuation. These historical dialogues act as part of the input. The key challenge of exploiting historical dialogues is that historical assistant text is usually considered unmanipulable and cannot be altered by the attacker. Existing multi-turn jailbreak studies [19], [20] remain confined to user-prompt manipulation. Given the significant impact of historical dialogues on an LLM's responses, we explore the research question: *How can historical dialogue manipulation be made entirely and practically, and how can it be utilized to enhance jailbreak attacks?*

A. Our Contributions

Our research takes the first step toward exploring how attacker-crafted historical dialogues can be leveraged to enhance jailbreaking attacks. We introduce a novel black-box jailbreak paradigm named DIA (Dialogue Injection Attack), which exploits the structural design of LLMs' chat templates

Received 30 April 2025; revised 24 November 2025 and 30 December 2025; accepted 15 January 2026. Date of publication 26 January 2026; date of current version 3 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302441, in part by the Key Research and Development Program Project of Ningbo under Grant 2025Z029, in part by the Open Fund of the Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, in part by the Zhejiang University Education Foundation Qizhen Scholar Foundation, and in part by the Information Technology Center of Zhejiang University and the Supercomputing Center of Hangzhou City University. The associate editor coordinating the review of this article and approving it for publication was Dr. Tianwei Zhang. *(Corresponding author: Chengkun Wei.)*

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University, and the Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, Hefei, China.

Wenlong Meng, Fan Zhang, Wendao Yao, Zhenyuan Guo, Chengkun Wei, and Wenzhi Chen are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: mengwl@zju.edu.cn; fan1.22@intl.zju.edu.cn; wendao.22@intl.zju.edu.cn; zhenyuanguo@zju.edu.cn; weichengkun@zju.edu.cn; chenwz@zju.edu.cn).

Yuwei Li is with the College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China (e-mail: liyuwei@nudt.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3657898

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

对话注入攻击：通过上下文操控破解 LLMs

Wenlong Meng , Fan Zhang, Wendao Yao, Zhenyuan Guo, Yuwei Li , Chengkun Wei , and Wenzhi Chen , Member, IEEE

摘要—大型语言模型（LLMs）在广泛应用中展现出显著效用；然而，其部署受到安全漏洞的困扰，尤其是破解攻击。这些攻击通过设计对抗性提示来操控 LLMs 生成有害或不道德的内容。尽管当前关于破解攻击的研究大多集中于单轮交互，但很大程度上忽略了历史对话对模型行为的影响。虽然近期研究探索了多轮破解攻击，但它们通常假设攻击者只能操控用户提示。相比之下，我们强调攻击者还可以控制模型的前期输出。为此，我们引入了对话注入攻击（Dialogue Injection Attack, DIA），这是一种利用伪造对话历史来增强破解效果的新范式。DIA 在黑盒环境下运行，仅需访问聊天 API 或了解 LLMs 的聊天模板即可。我们提出了两种构建对抗性历史对话的方法：一种适应灰盒预填充攻击，另一种利用延迟响应。 我们的实验表明，DIA 在最近的 LLMs（包括 Llama-3.1 和 GPT-4o）上实现了最先进的攻击成功率。此外，我们展示了 DIA 可以绕过 6 种不同的防御机制，突显了其鲁棒性。代码可在 https://github.上获取。com/meng-wenlong/DIA

索引词—越狱攻击，对话注入，安全对齐。

I. I

大型语言模型（LLMs）如 GPT-4[1]和 PaLM[2]的出现彻底改变了自然语言处理领域，在自然语言生成方面展现出强大的能力。由于它们强大的生成能力，LLMs 已被应用于众多内容生成应用中，包括代码生成器[3]、翻译者[4]，以及个性化推荐系统[5]。在这些应用中，聊天机器人是最受欢迎的，例如 OpenAI ChatGPT 和 Google Bard。由于 LLMs 在包含合规和非合规内容的广泛网络语料库上进行了训练，因此在提示时可能会生成有害内容[6][7]。为了减轻这些风险，LLM 开发者采用如人类反馈强化学习（RLHF）[8]等对齐技术，以确保输出符合人类伦理标准。这种对齐旨在确保 ChatGPT 等 LLMs 遵守伦理规范并促进建设性互动。

然而，集成到 LLMs 中的安全机制并非万无一失。安全和隐私问题在 AI 驱动系统中已被广泛研究，包括图神经网络[9]、语义通信[10]和私有信息检索[11]。LLMs 面临越狱攻击，攻击者会制作专门的提示来诱使 LLMs 产生有害或不道德的输出[12]，[13]。这些攻击采用多种技术，如催眠[14]、敏感词伪装[15]和时态转换[16]。这些被操控的输出可能促进进一步的恶意活动，包括从训练数据中提取个人信息[17]或创建恶意软件[18]。研究这些越狱攻击对于开发人员增强 LLM 应用的安全性以及改进与安全标准的对齐至关重要。

当前关于越狱方法的研究主要集中于单轮交互，其中所有对抗性内容都被压缩到单个提示中。然而，对话历史对塑造 LLM 的输出具有重要影响，尤其是在聊天机器人应用中。现有的 LLM 聊天机器人使用聊天模板将历史对话与当前提示连接起来，然后再提交给后端 LLM 进行续写。这些历史对话作为输入的一部分。利用历史对话的关键挑战在于，历史助手文本通常被认为是不可操控的，攻击者无法对其进行修改。现有的多轮越狱研究[19]、[20]仍然局限于用户提示操控。鉴于历史对话对 LLM 响应的显著影响，我们探讨了以下研究问题：如何使历史对话操控完全且实际可行，以及如何利用它来增强越狱攻击？

A. 我们的贡献

我们的研究迈出了探索如何利用攻击者构建的历史对话来增强越狱攻击的第一步。我们引入了一种名为 DIA（对话注入攻击）的新型黑盒越狱范式，该范式利用了 LLMs 聊天模板的结构设计

收到日期：2025 年 4 月 30 日；修改日期：2025 年 11 月 24 日和 2025 年 12 月 30 日；接受日期：2026 年 1 月 15 日。发表日期：2026 年 1 月 26 日；当前版本日期：2026 年 2 月 3 日。本研究部分由国家自然科学基金（项目编号：62302441）资助，部分由宁波市重点研发计划项目（项目编号：2025Z029）资助，部分由安徽省网络空间安全态势感知与评估重点实验室开放基金资助，部分由浙江大学教育基金会启真人才基金资助，部分由浙江大学信息技术中心及杭州城市学院超算中心资助。本文审稿协调编辑及发表批准人为张天伟博士。 通讯作者：魏成坤。）

这项研究涉及人类受试者或动物。所有伦理和实验程序及方案已获得浙江大学生物医学工程与仪器科学学院伦理委员会和安徽省合肥市网络安全态势感知与评估重点实验室的批准。

Wenlong Meng, Fan Zhang, Wendao Yao, Zhenyuan Guo, Chengkun Wei, and Wenzhi Chen are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: mengwl@zju.edu.cn; fan1.22@intl.zju.edu.cn; wendao.22@intl.zju.edu.cn; zhenyuanguo@zju.edu.cn; weichengkun@zju.edu.cn; chenwz@zju.edu.cn).

李宇伟就职于国防科技大学电子工程学院，地址为中国安徽省合肥市 230037（邮箱：liyuwei@nudt.edu.cn）。

数字对象标识符 10.1109/TIFS.2026.3657898

1556-6021 © 2026 IEEE。保留所有权利，包括文本和数据挖掘的权利，以及人工智能和类似技术的培训权利。允许个人使用，但再发布/重新分发需要 IEEE 的许可。
更多信息请参阅 https://www.ieee.org/publications/rights/index.html。
授权使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

to inject deceptive dialogues into the input. We show that an attacker, knowing the LLM's chat template, can use the structure of the template to insert arbitrary historical dialogues into the input (Section III-B), including assistant text and system text, which is usually considered unrealistic in black-box scenarios. Furthermore, we show how attackers can reduce the reliance on chat template knowledge by employing a *template inference attack* to deduce which chat template the victim LLM is using (Section III-C). Overall, DIA provides a novel attack vector to jailbreak by constructing a deceptive dialogue history.

Based on the DIA paradigm, we propose two methods for constructing adversarial dialogues, namely DIA-I and DIA-II. DIA-I is based on *prefilling attacks*, which place an affirmative prefix at the beginning of the assistant's response. We adapt this approach to black-box scenarios using a "continue" command and design an automated module to generate affirmative beginnings. DIA-II is based on a new finding that deferred malicious responses have a higher log-likelihood than those generated immediately after the prompt. We persuade the LLM to perform a word substitution task, achieving both answer deferral and malicious content disguise. In addition, we develop a prompt rewrite algorithm to enhance prompt diversity in multi-query scenarios. After 10 queries, our DIA achieves an attack success rate of 0.89 on Llama-3.1-8B and 0.82 on GPT-4o, evaluated on AdvBench.

In summary, our contributions are outlined as follows:

- We present a novel black-box jailbreak attack paradigm that leverages historical dialogues to enhance jailbreak attack performance. We provide the historical dialogue manipulation method that can inject arbitrary attacker-crafted dialogues, with the only requirement of victim LLMs' chat template. Furthermore, we introduce the template inference attack to infer LLMs' chat templates.
- Building on our proposed paradigm, we develop two methods for constructing adversarial dialogues: DIA-I and DIA-II. DIA-I adapts prefilling attacks to black-box scenarios, while DIA-II employs word substitution tasks to delay the delivery of actual answers. Additionally, we introduce a prompt rewrite algorithm designed to enhance the effectiveness of multi-query attacks.
- We evaluate DIA-I and DIA-II across three open-source jailbreak benchmarks against 10 LLMs, comparing their effectiveness with four state-of-the-art black-box jailbreak attacks. The experimental results demonstrate that our attacks achieved superior attack success rates on recent LLMs, e.g., 0.89 on Llama-3.1-8B and 0.82 on GPT-4o as tested by AdvBench. We also assess the robustness of DIA-I and DIA-II in penetrating defenses. Among five defenses evaluated, DIA achieves an average defense pass rate of 0.65.

II. Preliminaries

A. Large Language Models

Modern LLMs, like GPT-4 [1], Llama [21], Gemma [22], etc., are decoder-only Transformer models trained on next token prediction tasks. The training of LLMs consists of two phases: *pre-training* and *post-training*. In the pre-training phase, the model is trained on a vast corpus of text data, learning to understand and generate language by predicting next words. The pre-training phase is unsupervised, meaning it does not require labeled data. Given the sequence $X = \{x_t\}_1^T$, the pre-training loss for an LLM $\pi(\cdot;\theta)$ is

$$\mathcal{L} = -\sum_{t=1}^{T} \log \pi(x_t \mid x_{<t};\theta).$$

The post-training phase aims to enhance instruction-following capabilities. In this phase, training data is split into prompts and responses, which are alternately concatenated by a chat template into a continuous text block. Typically, the post-training loss is only computed on the final response [23], [24].

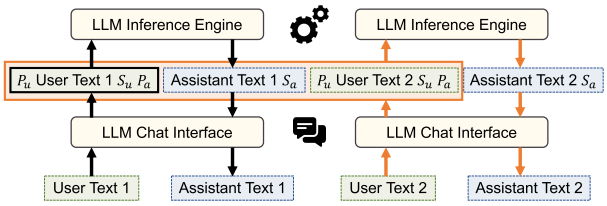


Fig. 1. LLM chat pipeline.

B. LLM Chat Pipeline

The fundamental ability of LLMs is to predict the next word according to previous input words. To handle chat tasks, LLMs utilize a chat template to integrate historical dialogues with the user prompt into a continuous text. Different types of LLMs often employ distinct chat templates. Despite their variations, modern chat templates typically adhere to a similar structure, wherein prefixes and suffixes are added to the text corresponding to different roles. For instance, Llama-3 and Gemma-2 exemplify this practice, as shown in Table I. In chat tasks, the roles generally include a user and an assistant, with many models (e.g., Llama-3 and Llama-2) also introducing a system entity to prioritize settings for the LLM. Using a chat template, a set of dialogue messages can be converted to the following text:

$$X = P_s\langle \text{system_text}\rangle S_s$$
$$[P_u\langle \text{user_text}\rangle S_u P_a\langle \text{assistant_text}\rangle S_a] \times n,$$

where n represents n rounds of conversation.

An LLM chat service comprises a chat interface as the frontend and an inference engine as the backend. The chat interface receives user prompts, wraps them with a chat template, and then sends them to the inference engine. After the inference engine generates the response, the chat interface then delivers the results back to the user. Figure 1 depicts how a chat service processes users' requests. A chat interface could be a WebUI (e.g., OpenAI ChatGPT or Google Gemini) or an API (e.g., OpenAI API or Google Gemini API). An inference engine, like vLLM [25] or Ollama [26], is designed to meet latency-related Service Level Objectives (SLOs). In Figure 1, during the inference stage of the second round chat, the prompt and response from the first round are concatenated to user text in the second round, ensuring the LLM has memory of the previous conversation.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

将欺骗性对话注入输入中。我们证明，攻击者如果知道 LLM 的聊天模板，可以利用模板的结构将任意的历史对话插入输入中（第三节 B），包括助手文本和系统文本，这在黑盒场景中通常被认为是不现实的。此外，我们展示了攻击者如何通过采用模板推理攻击来减少对聊天模板知识的依赖，从而推断受害者 LLM 正在使用哪个聊天模板（第三节 C）。总体而言，DIA 通过构建欺骗性对话历史提供了一种新的越狱攻击向量。

基于 DIA 范式，我们提出了两种构建对抗性对话的方法，即 DIA-I 和 DIA-II。DIA-I 基于预填充攻击，在助手回复的开头放置一个肯定的前缀。我们使用“继续”命令将这种方法应用于黑盒场景，并设计了一个自动模块来生成肯定的起始部分。DIA-II 基于一个新发现，即延迟恶意响应比在提示后立即生成的响应具有更高的对数似然。我们说服 LLM 执行一个词语替换任务，从而实现回答延迟和恶意内容伪装。此外，我们开发了一种提示重写算法，以增强多查询场景中的提示多样性。在 10 次查询后，我们的 DIA 在 Llama-3.1-8B 上达到了 0.89 的攻击成功率，在 GPT-4o 上达到了 0.82，这些数据是在 AdvBench 上评估得出的。

总之，我们的贡献概述如下：

- 我们提出了一种利用历史对话增强越狱攻击性能的新型黑盒越狱攻击范式。我们提供了历史对话操纵方法，该方法能够注入任意攻击者制作的对话，唯一要求是受害者 LLMs 的聊天模板。此外，我们介绍了模板推理攻击以推断 LLMs 的聊天模板。
- 基于我们提出的范式，我们开发了两种构建对抗性对话的方法：DIA-I 和 DIA-II。DIA-I 将预填充攻击适应于黑盒场景，而 DIA-II 采用词替换任务来延迟实际答案的交付。此外，我们引入了一种提示重写算法，旨在增强多查询攻击的有效性。
- 我们评估了 DIA-I 和 DIA-II 在三个开源越狱基准测试中的表现，针对 10 个 LLMs，将其有效性与四种最先进的黑盒越狱攻击进行了比较。实验结果表明，我们的攻击在最新的 LLMs 上取得了更高的攻击成功率，例如在 AdvBench 测试中，Llama-3.1-8B 达到了 0.89，GPT-4o 达到了 0.82。我们还评估了 DIA-I 和 DIA-II 穿透防御的鲁棒性。在评估的五种防御中，DIA 实现了平均 0.65 的防御通过率。

II. P

A. 大型语言模型

现代 LLMs，如 GPT-4 [1]、Llama [21]、Gemma [22]等，是仅使用解码器的 Transformer 模型，在下一词预测任务上进行训练。LLMs 的训练包括

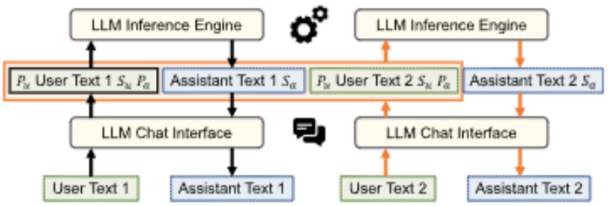


图 1. LLM 聊天流程。

两个阶段：预训练和后训练。在预训练阶段，模型在大量文本数据上进行训练，通过预测下一个词来学习理解和生成语言。预训练阶段是无监督的，这意味着它不需要标记数据。对于序列 X = {x}，LLM π (• ; θ)的预训练损失为

$$L = -\sum_{t=1}^{X^T} \log \pi (x| x; \theta) .$$

预训练阶段旨在提升指令跟随能力。在此阶段，训练数据被分为提示和响应，通过聊天模板交替连接成一个连续文本块。通常，预训练损失仅在最终响应上计算[23], [24]。

B. LLM 聊天流程

LLM 的基本能力是根据先前的输入词预测下一个词。为了处理聊天任务，LLM 利用聊天模板将历史对话与用户提示集成成一个连续文本。不同类型的 LLM 常常采用不同的聊天模板。尽管存在差异，现代聊天模板通常遵循相似结构，即在对应不同角色的文本前后添加前缀和后缀。例如，Llama-3 和 Gemma-2 体现了这一做法，如表 I 所示。在聊天任务中，角色通常包括用户和助手，许多模型（如 Llama-3 和 Llama-2）还引入系统实体以优先设置 LLM。使用聊天模板，一组对话消息可以转换为以下文本：

$$X = P〈系统文本〉S [P〈用户文本〉S P〈助手文本〉S] \times n,$$

其中 n 表示 n 轮对话。

LLM 聊天服务由前端聊天界面和后端推理引擎组成。聊天界面接收用户提示，用聊天模板将其封装，然后发送给推理引擎。推理引擎生成响应后，聊天界面再将结果返回给用户。图 1 描绘了聊天服务如何处理用户请求。聊天界面可以是 WebUI（例如 OpenAI ChatGPT 或 Google Gemini）或 API（例如 OpenAI API 或 Google Gemini API）。推理引擎，如 vLLM [25] 或 Ollama [26]，旨在满足与延迟相关的服务等级目标（SLOs）。在图 1 中，在第二轮对话的推理阶段，第一轮的提示和响应被连接到第二轮的用户文本中，确保 LLM 记得之前的对话。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

TABLE I
EXAMPLES OF LLM CHAT TEMPLATES STRUCTURE. LLAMA-3 AND GEMMA-2 USE DIFFERENT PREFIXES TO INDICATE THE ROLES

Component	Description	Llama-3	Gemma-2
P_s	Prefix of system text	<|start_header_id|>system<|end_header_id|>\n\n	-
S_s	Suffix of system text	<|eot_id|>	-
P_u	Prefix of user text	<|start_header_id|>user<|end_header_id|>\n\n	<start_of_turn>user\n
S_u	Suffix of user text	<|eot_id|>	<end_of_turn>\n
P_a	Prefix of assistant text	<|start_header_id|>assistant<|end_header_id|>\n\n	<start_of_turn>model\n
S_a	Suffix of assistant text	<|eot_id|>	<end_of_turn>\n

C. Safety Alignment and Jailbreak Attacks

1) Safety Alignment: Due to exposure to a broad range of texts during pre-training, including both safe and unsafe sources, LLMs can generate unethical content. To ensure the safety of the content produced by LLMs, developers often incorporate safety alignment during the post-training phase. Common safety alignment methods include Supervised Fine-Tuning (SFT), Reinforcement Learning from Human Feedback (RLHF) [8], and Direct Preference Optimization (DPO) [23]. The goal of these methods is to enable LLMs to refuse to respond to harmful inquiries while still effectively answering benign questions.

2) Jailbreak Attacks: Jailbreak attacks refer to techniques that exploit vulnerabilities in LLMs to bypass their built-in safety and ethical guidelines. Depending on the threat model, existing jailbreak attacks can be classified into: *white-box*, *gray-box*, and *black-box* categories. White-box attacks require access to the architecture and parameters of the victim LLM for training purposes. For example, GCG [27] optimizes an adversarial suffix through a combination of gradient heuristics and greedy search. Gray-box attacks also need access to the model but do not require training. A common type of gray-box attack is the prefilling attack [28], [29], which simply prefills an LLM to start its response with an affirmative beginning, such as "*Sure, I'd be happy to help.*". Since general LLM chat services do not allow users to manipulate model responses directly, prefilling attacks require the attacker to perform inference locally. Black-box attacks, on the other hand, do not require any knowledge of the victim model. Current black-box jailbreak attacks typically use carefully crafted prompts to exploit vulnerabilities in safety alignment. As developers cannot address all potential harmful scenarios during safety alignment, there remain backdoors that can circumvent safety guidelines and trigger harmful behaviors. For instance, PromptInject [30] shows prompt injection can cause LLMs to ignore system instructions and produce adversary-desired outputs.

3) Limitations of Existing Jailbreak Attacks: White-box and gray-box attacks require access to a model's parameters, which is often impractical. Additionally, these types of attacks demand significant computational power from attackers, limiting their widespread use and harm. Although black-box jailbreak attacks are easier to perform, they are easier to patch by RLHF. After disclosure of black-box jailbreak attacks, LLM developer can add these jailbreak prompts to their alignment dataset, which can help the LLM detect and thwart such attacks. In Section V-B, we observe that two state-of-the-art black-box jailbreak attacks, DeepInception and DRA, experience ASR degradation by 67% and 99% on Llama-3.1-8B, respectively, compared to their performance on Llama-2-7B. Crescendo [19] opens the door to multi-turn jailbreaks by adaptively designing the next prompt based on model's response. However, Crescendo entails manual effort because aligned models tend to resist generating Crescendo jailbreaks.

III. ATTACK INTUITION

Motivation: As discussed in Section II-B, the current user prompt is concatenated with previous dialogues to form the complete input for inference. As a result, historical dialogue plays a crucial role in shaping the model's response. *An LLM, when presented with the same user prompt but different historical dialogues, will exhibit variations in its output distribution*. This hypothesis can be formalized as:

$$\pi\left(X \mid H; \theta\right) \neq \pi\left(X \mid H'; \theta\right),$$

where X is the user prompt of current round, H and H' are history dialogues, $H \neq H'$. Existing black-box jailbreak attacks typically assume that H is empty, neglecting the impact of history dialogues. In this section, we explore the practicality of using historical dialogues to implement jailbreak attacks.

A. Threat Model

1) Attacker's Goal: We adhere to the traditional objective of jailbreak attacks [13], which is to manipulate a victim LLM into responding harmfully and accurately to a malicious prompt. This means that the LLM should not only avoid refusing to respond but also must provide a reasonable output that fulfills the requirements of the prompt.

2) Attacker's Capability: In this paper, we consider the strict black-box scenario where the adversary lacks access to the architecture, parameters, and training data of the victim model, rendering it impossible to reproduce the model. The adversary can interact with the victim LLM using two methods: 1) via a chat API, or 2) through a WebUI. In the second case, we assume the adversary has knowledge of the victim LLM's chat template. Nevertheless, this assumption can be relaxed if the victim LLM uses a publicly available chat template. The adversary could use the template inference attack, as detailed in Section III-C, to deduce the specific template used.

B. Historical Dialogue Manipulation

1) Dialogue Injection: Manipulating historical dialogue is straightforward when adversaries have access to the chat API, as they can directly set previous assistant responses. Typically, completely controlling responses through a WebUI is deemed impossible since LLMs are probabilistic models and their

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

表 I

ELLM CTS. L-3 G-2 UDPIR

Component	Description	Llama-3	Gemma-2
P_s	Prefix of system text	<|start_header_id|>system<|end_header_id|>\n\n	-
S_s	Suffix of system text	<|eot_id|>	-
P_u	Prefix of user text	<|start_header_id|>user<|end_header_id|>\n\n	<start_of_turn>user\n
S_u	Suffix of user text	<|eot_id|>	<end_of_turn>\n
P_a	Prefix of assistant text	<|start_header_id|>assistant<|end_header_id|>\n\n	<start_of_turn>model\n
S_a	Suffix of assistant text	<|eot_id|>	<end_of_turn>\n

C. 安全对齐和破解攻击

1) 安全对齐：由于在预训练阶段接触了广泛的各种文本，包括安全和不安全的来源，LLMs 可能会生成不道德的内容。为确保 LLMs 生成的内容安全，开发人员通常在后训练阶段加入安全对齐。常见的安全对齐方法包括监督微调（SFT）、人类反馈强化学习（RLHF）[8]和直接偏好优化（DPO）[23]。这些方法的目标是使 LLMs 能够拒绝回答有害的询问，同时仍然有效地回答良性问题。

2) 脱逃攻击：脱逃攻击是指利用 LLM 漏洞以绕过其内置的安全和伦理指南的技术。根据威胁模型，现有的脱逃攻击可分为：白盒、灰盒和黑盒三类。白盒攻击需要访问受害者 LLM 的架构和参数进行训练。例如，GCG [27] 通过梯度启发式和贪婪搜索的组合优化对抗性后缀。灰盒攻击也需要访问模型，但不需要训练。常见的灰盒攻击类型是预填充攻击 [28], [29]，它通过预填充 LLM 使其响应以肯定的开始，例如“当然，我很乐意帮忙”。由于通用 LLM 聊天服务不允许用户直接操纵模型响应，预填充攻击需要攻击者在本地进行推理。另一方面，黑盒攻击则不需要了解受害者模型。当前的黑盒脱逃攻击通常使用精心设计的提示来利用安全对齐中的漏洞。 由于开发者在安全对齐过程中无法应对所有潜在的有害场景，仍然存在可以绕过安全指南并触发有害行为的后门。例如，PromptInject [30] 表明提示注入可以使 LLM 忽略系统指令并产生攻击者期望的输出。

3) 现有越狱攻击的局限性：白盒
以及灰盒攻击需要访问模型的参数，这通常不切实际。此外，这类攻击需要攻击者投入大量的计算资源，限制了其广泛使用和危害。尽管黑盒越狱攻击更容易实施，但它们更容易通过 RLHF 进行修补。在黑盒越狱攻击披露后，LLM 开发者可以将这些越狱提示添加到他们的对齐数据集中，这有助于 LLM 检测并挫败此类攻击。在第五节-B 中，我们观察到两种最先进的黑盒越狱攻击，DeepInception 和 DRA，在 Llama-3.1-8B 上的 ASR 退化分别为 67% 和 99%。

分别地，与它们在 Llama-2-7B 上的表现相比。Crescendo [19] 通过根据模型的响应自适应地设计下一个提示，为多轮越狱打开了大门。然而，Crescendo 需要人工操作，因为对齐的模型往往倾向于抵抗生成 Crescendo 越狱。

III. AI

动机：如第二节-B 所述，当前的用户提示与之前的对话连接起来，形成用于推理的完整输入。因此，历史对话在塑造模型响应中起着至关重要的作用。当向 LLM 展示相同用户提示但不同的历史对话时，它的输出分布将表现出差异。这个假设可以被形式化为：

$$\pi(X \mid H; \theta), \pi \quad X \mid H; \theta \quad \boxtimes,$$

其中 X 是当前回合的用户提示，H 和 Hare 历史对话，H ，H。现有的黑盒越狱攻击通常假设 H 为空，忽略了历史对话的影响。在本节中，我们探讨了使用历史对话实施越狱攻击的实用性。

A. 威胁模型

1) 攻击者的目标：我们遵循越狱攻击的传统目标[13]，即操纵受害 LLM 对恶意提示做出有害且准确的回应。这意味着 LLM 不仅要避免拒绝回应，还必须提供满足提示要求的合理输出。

2) 攻击者的能力：在本文中，我们考虑严格的黑盒场景，其中攻击者无法访问受害模型的架构、参数和训练数据，导致无法复制该模型。攻击者可以通过两种方法与受害 LLM 进行交互：1) 通过聊天 API，或 2) 通过 WebUI。在第二种情况下，我们假设攻击者了解受害 LLM 的聊天模板。然而，如果受害 LLM 使用公开可用的聊天模板，则此假设可以放宽。攻击者可以使用第三节 C 中详细描述的模板推理攻击，来推断所使用的具体模板。

B. 历史对话操控

1) 对话注入：当攻击者可以访问聊天 API 时，操控历史对话很简单，因为他们可以直接设置先前的助手回复。通常认为通过 WebUI 完全控制回复是不可能的，因为 LLM 是概率模型，它们的

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

TABLE II
INVESTIGATION INTO INPUT SANITIZATION IN OPEN-SOURCE LLM WEBUIS. SORTED BY STAR COUNT IN DESCENDING ORDER

WebUI	Default sanitize	Optional sanitize	#Stars
OpenWebUI [34]	✗	✓	116k
LobeChat [35]	✗	✗	67.9k
Text Generation WebUI [36]	✗	✗	45.4k
StreamLit [37]	✗	✓	42.3k
Gradio [38]	✗	✓	40.6k
Jan [39]	✗	✗	39.4k
ChatBox [40]	✗	✗	37.5k
LibreChat [41]	✗	✗	31.8k
SillyTavern [42]	✗	✗	20.1k
Hugging Face ChatUI [43]	✗	✗	10.3k
BionicGPT [44]	✗	✗	2.3k

outputs cannot be predetermined. However, we propose that an adversary could fully control previous assistant responses using a technique we refer to as *dialogue injection*. This method is inspired by *SQL injection*, where malicious SQL statements are inserted into an entry field to be executed. By utilizing a WebUI as the entry point, an adversary can strategically inject crafted responses by surrounding them with S_uP_a and S_aP_u. Formally, if the adversary wants to insert a crafted conversation $\left[\langle \text{user_text}\rangle^1, \langle \text{assistant_text}\rangle^1\right]$ before $\langle \text{user_text}\rangle^2$, they can compose their user prompt as:

$$X_{adv} = \langle \text{user_text}\rangle^1 S_u P_a \langle \text{assistant_text}\rangle^1 S_a P_u \langle \text{user_text}\rangle^2. \quad (1)$$

The WebUI will prepend $P_s\langle \text{system_text}\rangle S_s$ and append S_uP_a to X_{adv}. In this case, the LLM inference engine receives a context identical to the round 2 illustrated in Figure 1. The difference is that while the assistant text in Figure 1 is generated by the LLM, $\langle \text{assistant_text}\rangle^1$ in Equation 1 is fabricated by the attacker. The attacker can also insert multiple conversations into the inputs using the methods outlined in Equation 1. Thanks to advances in increasing the input sequence length of LLMs [31], [32], [33], current LLM chat WebUIs accept very long inputs, so the attacker need not worry about exceeding input length limitations when inserting dialogues.

One potential defense against dialogue injection is to detect or remove special tokens from the user input. We investigate 11 popular open-source LLM WebUIs on GitHub to see if they sanitize user inputs. The results of this investigation are summarized in Table II. Unfortunately, none of the WebUIs implement input sanitization by default, and only three provide an optional sanitization feature. For example, OpenWebUI [34] provides a filter function plugin to modify inputs and outputs. The remaining WebUIs would require intrusive code modifications to support special-token detection or filtering. Without additional protective measures from the LLM developers, most WebUIs are vulnerable to dialogue injection.

By leveraging chat APIs or dialogue injection, certain gray-box jailbreak attacks can also be executed in black-box scenarios.

2) Empowering Prefilling Attack: This attack prefills the first few tokens with a non-refusal prefix at the beginning of the inference. Prefilling attack exploits the characteristics of shallow safety alignment [45]. Safety alignment techniques often constrain only the probability distribution of the initial few tokens in an output. Attackers can circumvent these alignments by setting the initial tokens of a response. In black-box scenarios, we cannot alter the outputs of the LLM directly. However, we can manipulate the model into believing it has already generated some content by injecting history dialogues. Then, we can make the LLM continue its previous output. The corresponding adversarial prompt for the WebUI interface is

$$X_{adv} = \langle \text{user_text}\rangle S_u P_a \langle \text{prefill}\rangle S_a P_u \langle \text{continue_cmd}\rangle.$$

3) Empowering System Prompt Replacement Attack: System prompts have the highest priority. If attackers can make the victim LLM lower its guard via a system prompt, they can increase the jailbreak success rate. With access to the chat API, attackers can set their own system prompts. As for WebUI access, attackers can also replace $\langle \text{system_text}\rangle$ with $\langle \text{system_text}\rangle'$ using the adversarial input below

$$X_{adv} = \langle \text{user_text}\rangle^1 S_u P_a \langle \text{assistant_text}\rangle^1 S_a P_s \langle \text{system_text}\rangle' S_s P_a \langle \text{user_text}\rangle^2.$$

Attackers can prompt the LLM to forget previous commands in $\langle \text{system_text}\rangle'$ to negate the effects of $\langle \text{system_text}\rangle$.

C. Template Inference Attack

A practical concern with dialogue injection is that it requires the attacker to have knowledge of the chat template. A chat template of an LLM can be arbitrary. However, LLM developers generally use post-trained models for secondary development since post-training requires substantial datasets and computational resources. It is challenging for individual developers and small companies to conduct high-quality post-training. It is also difficult for LLM developers to change the chat template of a post-trained model because template transfer can lead to performance degradation. In our experiments with the Gemma-2-2B model [22], we attempted a template transfer using 10,000 entries from NoRobots dataset [46]. We input the user prompts from NoRobots into Gemma-2-2B, collected the outputs, and then concatenated these prompts and outputs using the Llama-3 template to fine-tune Gemma-2-2B for one epoch. We observed that the AlpacaEval 2.0 score [47] of Gemma-2-2B decreased from 41.2 to 28.8, indicating that LLM developers cannot trivially alter the chat templates of post-trained models.

If an LLM developer uses an open-sourced chat template, we propose a method to infer which chat template is used by the LLM, named template inference attack (TIA). Assume an LLM utilizes one chat template, T_{inf}, from an open-source collection of chat templates $\mathcal{T} = \{T_n\}_1^N$. We propose a method to ascertain whether $T_i = T_{inf}$ or $T_j = T_{inf}$ by combining two templates, T_i and T_j, to create a probe. The probe prompt formed as:

$$X_P = \langle \text{user_text}\rangle S_{ui} P_{ai} S_{uj} P_{aj} \langle \text{assistant_text}\rangle S_{ai} P_{ui} S_{aj} P_{uj} \langle \text{repeat_cmd}\rangle, \quad (2)$$

where $\langle \text{repeat_cmd}\rangle$ is a command to ask the LLM to repeat its previous response. When $T_i = T_{inf}$, $S_{ui}P_{ai}$ and $S_{ai}P_{ui}$ will act as the template part. LLM will output $S_{uj}P_{aj}\langle \text{assistant_text}\rangle$. When $T_j = T_{inf}$, $S_{uj}P_{aj}$ and $S_{aj}P_{uj}$ will act as the template part. LLM will output

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

表 II

IIISO-SLLM
WUI. SSCDO

WebUI	Default sanitize	Optional sanitize	#Stars
OpenWebUI [34]	✗	✓	116k
LobeChat [35]	✗	✗	67.9k
Text Generation WebUI [36]	✗	✗	45.4k
StreamLit [37]	✗	✓	42.3k
Gradio [38]	✗	✓	40.6k
Jan [39]	✗	✗	39.4k
ChatBox [40]	✗	✗	37.5k
LibreChat [41]	✗	✗	31.8k
SillyTavern [42]	✗	✗	20.1k
Hugging Face ChatUI [43]	✗	✗	10.3k
BionicGPT [44]	✗	✗	2.3k

输出结果不能预先确定。然而，我们提出一种攻击者可以使用的技术，我们称之为对话注入，来完全控制先前的助手响应。这种方法受到 SQL 注入的启发，其中恶意 SQL 语句被插入到输入字段中以被执行。通过利用 WebUI 作为入口点，攻击者可以战略性地通过用 S 和 S P 包围来注入精心制作的响应。形式上，如果攻击者想要插入一个精心制作的对话

〈用户文本〉， 助手文本〉 ☒之前〈用户文本〉，他们可以将其用户提示编写为：

X=〈用户文本〉S P〈助手文本〉S P 〈用户文本〉. (1)

WebUI 将在 X 前面添加 P〈系统文本〉S，并在后面追加 S P。在这种情况下，LLM 推理引擎接收到的上下文与图 1 中所示的第二轮对话相同。不同之处在于，图 1 中的助手文本是由 LLM 生成的，而方程 1 中的〈助手文本〉是由攻击者伪造的。攻击者还可以使用方程 1 中概述的方法将多个对话插入输入中。得益于 LLM 输入序列长度增加的进展 [31], [32], [33]，当前的 LLM 聊天 WebUI 可以接受非常长的输入，因此攻击者在插入对话时无需担心超出输入长度限制。

一种潜在的防御对话注入的方法是从用户输入中检测或移除特殊标记。我们在 GitHub 上调查了 11 个流行的开源 LLM WebUI，以查看它们是否对用户输入进行清理。这项调查的结果总结在表 II 中。不幸的是，这些 WebUI 中没有一个默认实现输入清理，只有三个提供可选的清理功能。例如，OpenWebUI [34]提供了一个过滤函数插件来修改输入和输出。其余的 WebUI 需要侵入性的代码修改来支持特殊标记检测或过滤。如果没有 LLM 开发者的额外保护措施，大多数 WebUI 容易受到对话注入的攻击。

通过利用聊天 API 或对话注入，某些灰盒越狱攻击也可以在黑盒场景中执行。

2) 赋能预填充攻击：这种攻击预填充
在推理开始时，以非拒绝前缀开头的最初几个 token。预填充攻击利用了浅层安全对齐的特性[45]。安全对齐技术通常仅限制输出中最初几个 token 的概率分布

攻击者可以通过设置响应的初始标记来绕过这些对齐。在黑盒场景中，我们无法直接修改 LLM 的输出。然而，我们可以通过注入历史对话来操纵模型，使其相信它已经生成了一些内容。然后，我们可以让 LLM 继续其之前的输出。对应于 WebUI 界面的对抗性提示是

X=〈用户文本〉S P〈预填〉S P〈继续命令〉.

3) 赋能系统提示替换攻击：Sys-
文本提示具有最高优先级。如果攻击者可以通过系统提示让受害者 LLM 放松警惕，他们可以提高越狱的成功率。通过访问聊天 API，攻击者可以设置自己的系统提示。至于 WebUI 访问，攻击者也可以使用以下对抗性输入替换〈system text〉为〈system text〉。

X=〈用户文本〉S P〈助手文本〉S P 〈系统文本〉S P〈用户文本〉.

攻击者可以通过在〈系统文本〉中提示 LLM 忘记之前的命令来抵消〈系统文本〉的效果。

C. 模板推断攻击

对话注入的一个实际问题在于它需要攻击者了解聊天模板。LLM 的聊天模板可以是任意的。然而，LLM 开发者通常使用预训练模型进行二次开发，因为预训练需要大量数据集和计算资源。对个人开发者和小公司来说，进行高质量的预训练非常困难。此外，LLM 开发者也很难改变预训练模型的聊天模板，因为模板迁移可能导致性能下降。在我们的 Gemma-2-2B 模型实验中[22]，我们尝试使用 NoRobots 数据集[46]中的 10,000 条数据来进行模板迁移。我们将 NoRobots 中的用户提示输入到 Gemma-2-2B 中，收集输出，然后使用 Llama-3 模板将提示和输出连接起来，对 Gemma-2-2B 进行一个 epoch 的微调。我们观察到 Gemma-2-2B 的 AlpacaEval 2.0 得分[47]从 41.2 下降到 28.8，这表明 LLM 开发者不能轻易地改变预训练模型的聊天模板。

如果 LLM 开发者使用了一个开源的聊天模板，我们提出了一种推断 LLM 使用了哪个聊天模板的方法，称为模板推断攻击（TIA）。假设一个 LLM 从开源聊天模板集合 T = {T}中使用了其中一个聊天模板 T。我们提出了一种方法来确定 T 是否为 T 或 T，通过结合两个模板 T 和 T 来创建一个探测。探测提示的格式如下：

X=〈用户文本〉S PS P〈助手文本〉 S PS P〈重复命令〉， (2)

其中〈重复命令〉是一个命令，要求 LLM 重复其之前的响应。当 T=T 时，SP 和 SP 将作为模板部分。LLM 将输出 SP〈助手文本〉。当 T=T 时，SP 和 SP 将作为模板部分。LLM 将输出

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

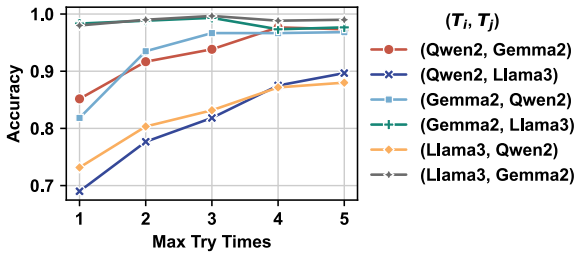


Fig. 2. Evaluation results of chat template inference attack. We test on three LLMs, using two of their chat templates to construct probes.

⟨assistant_text⟩$S_{ai}P_{ui}$. When $T_i \neq T_{inf}$ and $T_j \neq T_{inf}$, LLM will become confused, outputting either both $S_{uj}P_{aj}$ and $S_{ai}P_{ui}$ or neither. Let T_i and T_j be the chat templates for Gemma-2 and Llama-3, respectively. By substituting the values from Table I into Equation 2, we can obtain the following probe.

Template Inference Probe of GEMMA-2 and LLAMA-3

user text placeholder<end_of_turn>
<start_of_turn>model
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
assistant text placeholder<end_of_turn>
<start_of_turn>user
<|eot_id|><|start_header_id|>user<|end_header_id|>
Please repeat your previous response.

The intuition of our TIA is that if the attacker guesses the correct template, the LLM will interpret the instruction and output the expected content. To minimize false positives, we use a meaningless placeholder as the assistant text, since meaningful text could interfere with the LLM's output. However, with meaningless assistant text, the LLM consistently outputs "no previous response" if the probe contains only one template. We address this challenge by employing interleaved templates so that the assistant text includes template content. This approach also increases inference efficiency by allowing two templates to be inferred simultaneously.

In our experiments with Qwen-2, we observe that when $T_i = T_{inf}$, the presence of $S_{aj}P_{uj}$ occasionally disrupted the ⟨repeat_cmd⟩, complicating the LLM's ability to comprehend the repeat command. To address this issue, we develop a *swap-and-retry* method. Initially, we utilize Equation 2 to construct the probe. After N_{Tmax} attempts, if neither T_i nor T_j matches the inference, we swap their positions in Equation 2 to generate a revised probe, X'_{probe}. This probe is then used for another N_{Tmax} inference times.

1) Evaluation of TIA: We test TIA on three representative LLMs: Llama-3-8B, Gemma-2-9B, and Qwen-2-7B, each having distinct chat templates. We select two templates from them as T_i and T_j and apply TIA to infer each model 200 times. We report the inference accuracy versus max try times N_{Tmax} in Figure 2. As the max try times increase, the template inference accuracy gradually improves. We discover that when N_{Tmax} is set to 5, the inference accuracy can reach approximately 0.9 or higher. This indicates that an attacker only needs to query the WebUI 10 times to determine whether the target LLM's chat template is T_i or T_j.

2) n-Candidates Inference: To extend the inference process to multiple candidate templates, we define c as the number of template combinations that need to be queried. Each combination consists of two candidate templates. Let n denote the number of candidates and assume the correct template is among the candidates. When n is even, $c = \frac{n}{2}$; when n is odd, $c = \frac{n-1}{2}$. Using the swap-and-retry strategy, each combination requires $2N_{Tmax}$ queries and produces one of three possible outcomes: T_i, T_j, or neither, with an accuracy of p. The TIA process terminates once T_i or T_j is output. Assuming completing an inference needs N queries and C combinations, we can compute the expectation of N by

$$\begin{aligned}\mathbb{E}[N] &= 2N_{Tmax}\mathbb{E}[C] \\ &= 2N_{Tmax}\sum_{j=1}^{c} j\Pr(C=j) \\ &= 2N_{Tmax}\sum_{j=1}^{c}\sum_{i=1}^{j}\Pr(C=j) \\ &= 2N_{Tmax}\sum_{i=1}^{c}\sum_{j=i}^{c}\Pr(C=j) \\ &= 2N_{Tmax}\sum_{i=1}^{c}\Pr(C\geq i). \end{aligned} \tag{3}$$

In this way, we transform $\mathbb{E}(N)$ into a summation over tail probabilities. $C \geq i$ means that the first $i-1$ combinations all yield the result of neither. When n is even, the correct candidate has a probability of $\frac{i-1}{c}$ being among the first $i-1$ combinations and a probability of $\frac{c-i+1}{c}$ not being included. In this case, the tail probability is

$$\Pr(C\geq i) = \frac{c-i+1}{c}p^{i-1} + \frac{i-1}{c}p^{i-2}\frac{1-p}{2}. \tag{4}$$

Substituting Equation 4 into Equation 3 gives

$$\begin{aligned}\frac{\mathbb{E}[N]}{2N_{T_{\max}}} &= \frac{p^{c-1}\left(-p^2c+3p^2+2pc-p-c\right)}{2c(1-p)^2} \\ &\quad + \frac{-2pc-3p+2c+1}{2c(1-p)^2}, \quad n \bmod 2 = 0. \end{aligned} \tag{5}$$

When n is odd, the correct candidate has a probability of $\frac{2i-2}{2c+1}$ of being among the first $i-1$ candidates, and a probability of $\frac{2c+3-2i}{2c+1}$ not being included. Similarly, in this case,

$$\Pr(C\geq i) = \frac{2c+3-2i}{2c+1}p^{i-1} + \frac{2i-2}{2c+1}p^{i-2}\frac{1-p}{2}. \tag{6}$$

Substituting Equation 6 into Equation 3 gives

$$\frac{\mathbb{E}[N]}{2N_{T_{\max}}} = \frac{A+B}{(1-p)^2(2c+1)}, n \bmod 2 = 1, \tag{7}$$

where $A = (1-p)\left((2c+4)-(c+1)p^c-(c+3)p^{c+1}\right)$ and $B = -2\left(1-(c+2)p^{c+1}+(c+1)p^{c+2}\right)$.

Then, we compute the accuracy of inference under n-candidates. A successful prediction requires that all combinations from the first combination to the correct candidate are predicted successfully. When n is even, we get

$$\Pr(T=T_{inf}) = \sum_{k=1}^{c} p^k \cdot \frac{1}{c} = \frac{p\left(1-p^c\right)}{c(1-p)}, n \bmod 2 = 0. \tag{8}$$

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

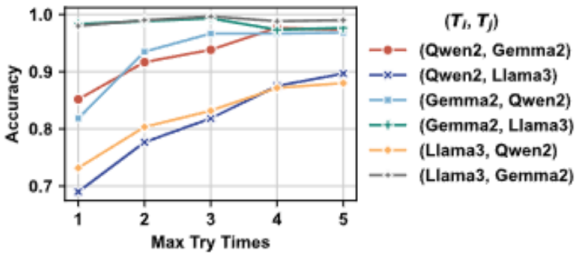


图 2. 聊天模板推理攻击的评估结果。我们在三个 LLM 上进行测试，使用其中两个 LLM 的聊天模板来构建探测。

〈助手文本〉S P. 当 T 时，T 和 T LLM 会变得困惑，输出 S P 和 S P 或都不输出。令 T 和 T 分别为 Gemma2 和 Llama-3 的聊天模板。通过将表 1 中的值代入方程 2，我们可以得到以下探测。

GEMMA-2 和 LLAMA-3 的模板推理探测

用户文本占位符<结束回合> <开始回合>模型<|eot id|><|开始头部 id|>助手<|结束头部 id|> 助手文本占位符<结束回合> <开始回合>用户<|eot id|><|开始头部 id|>用户<|结束头部 id|> 请重复你之前的回复。我们 TIA 的直觉是，如果攻击者猜对了正确的模板，LLM 将解释指令并输出预期内容。为了最小化误报，我们使用无意义的占位符作为助手文本，因为有意义文本可能会干扰 LLM 的输出。然而，使用无意义的助手文本时，如果探测内容只包含一个模板，LLM 始终输出“没有之前的回复”。我们通过采用交错模板来解决这个问题，以便助手文本包含模板内容。这种方法还通过允许同时推断两个模板来提高推理效率。

在我们的 Qwen-2 实验中，我们观察到当 T=T 时，S P 的存在偶尔会扰乱〈重复指令〉，使 LLM 难以理解重复指令。为解决这个问题，我们开发了一种交换并重试的方法。最初，我们利用公式 2 构建探测。在 N 次尝试后，如果 T 和 T 都不匹配推理结果，我们将它们在公式 2 中的位置进行交换以生成修正后的探测 X。然后使用这个探测进行另外 N 次推理。

1) TIA 评估：我们在三个具有不同聊天模板的代表性 LLM 上测试了 TIA：Llama-3-8B、Gemma-2-9B 和 Qwen-2-7B。我们从中选择两个模板作为 T 和 T，并应用 TIA 对每个模型进行 200 次推理。我们在图 2 中报告了推理准确率与最大尝试次数 N 的关系。随着最大尝试次数的增加，模板推理准确率逐渐提高。我们发现当 N 设置为 5 时，推理准确率可以达到约 0.9 或更高。这表明攻击者只需要查询 WebUI 10 次就能确定目标 LLM 的聊天模板是 T 还是 T。

2) n-候选推理：为了将推理过程扩展到多个候选模板，我们将 c 定义为需要查询的模板组合。每个组合由两个候选模板组成。用 n 表示候选数量，假设正确模板在候选中。当 n 为偶数时，c = ；当 n 为奇数时，c = 。使用交换重试策略，每个组合需要 2N 次查询，并产生三种可能结果之一：T、T 或都不是，准确率为 p。TIA 过程一旦输出 T 或 T 就会终止。假设完成一次推理需要 N 次查询和 C 个组合，我们可以通过计算来得到 N 的期望值。

$$\begin{aligned} E[N] &= 2NE[C] \\ &= 2N \sum_{j=1}^{c} j \Pr(C = j) \\ &= 2N \sum_{j=1}^{c} \sum_{i=1}^{j} \Pr(C = j) \\ &= 2N \sum_{i=1}^{c} \sum_{j=i}^{c} \Pr(C = j) \\ &= 2N \sum_{i=1}^{c} \Pr(C \geqslant i). \end{aligned} \tag{3}$$

这样，我们将 E(N)转化为对尾部概率的求和。C ⩾ i 表示前 i − 1 个组合都没有产生结果。当 n 为偶数时，正确候选在第一个 i − 1 个组合中的概率为 c 不被包含的概率。在这种情况下，尾部概率是

$$\Pr(C \geqslant i) = \frac{c - i + 1}{c} p + \frac{i - 1}{c} p \left(1 - p\right)^2. \tag{4}$$

将公式 4 代入公式 3 得到

$$\frac{E[N]}{2N} = \frac{p\left(-pc + 3p + 2pc - p - c\right)}{2c(1-p)} + \frac{-2pc - 3p + 2c + 1}{2c(1-p)}, \; n \bmod 2 = 0. \tag{5}$$

当 n 为奇数时，正确的候选者有 2 的概率出现在前 i − 1 个候选者中，并且有 $2c^{3-2i}$ 2c+1 未被包含。类似地，在这种情况下，

$$\Pr(C \geqslant i) = \frac{2c + 3 - 2i}{2c + 1} p + \frac{2i - 2}{2c + 1} p \left(1 - p\right)^2. \tag{6}$$

将公式 6 代入公式 3 得到

$$\frac{E[N]}{2N} = \frac{A + B}{(1-p)(2c+1)}, \; n \bmod 2 = 1, \tag{7}$$

其中 $A = (1-p)\left((c+2)p + (c+1)\right)$ 和 $B = -2\left((c+4) - (c+1)p - (c+3)p\right)$。

然后，我们计算在 n 个候选情况下的推理准确率。成功的预测要求从第一个组合到正确候选的所有组合都被成功预测。当 n 为偶数时，我们得到

$$\Pr(T = T) = \sum_{k=1}^{c} p \cdot \frac{1}{c} = \frac{p(1-p)}{c(1-p)}, \; n \bmod 2 = 0. \tag{8}$$

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

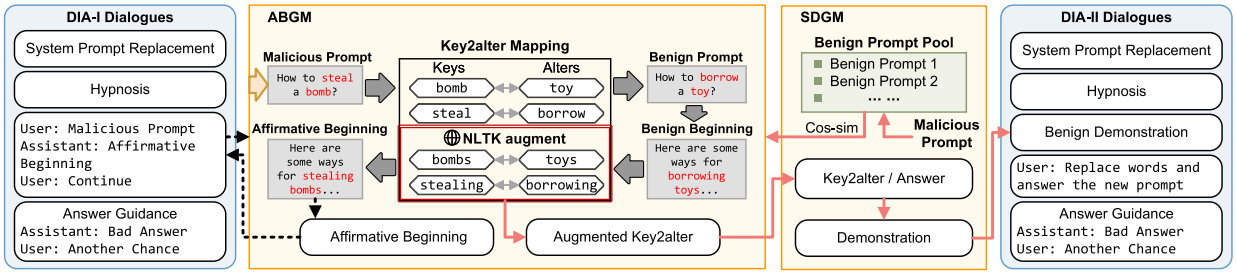


Fig. 3. Dialogue construction processes of DIA-I and DIA-II. This figure illustrates the composition of DIA-I and DIA-II dialogues on the left and right sides, respectively. DIA-I prompts the victim LLM to continue its previous affirmative response, while DIA-II engages the victim LLM in a word substitution task, subsequently answering prompts with the substituted words. DIA-I utilizes an Affirmative Beginning Generation Module (ABGM) to automate the creation of affirmative beginnings of malicious responses. DIA-II employs a Similar Demonstration Generation Module (SDGM) to generate benign demonstrations.

When n is odd, we get

$$\Pr(T = T_{inf}) = \sum_{k=1}^{c} p^k \cdot \frac{2}{2c+1} + p^c \cdot \frac{1}{2c+1} = \frac{2p - 3p^{c+1} + p^c}{(2c+1)(1-p)}, n \bmod 2 = 1. \quad (9)$$

We review the top 100 models on Hugging Face Open LLM Leaderboard [48] and find that only five distinct chat templates are in use, namely ChatML, Mistral, OpenBuddy, Llama-V3, and Phi-V4. As shown in Figure 2, detection accuracy p exceeds 0.9 in most cases when $N_{Tmax} = 5$, so we set $p = 0.9$. We note that the p may vary across template pairs. In our experiments, p exceeds 0.9 in most cases when $N_{Tmax} = 5$, so we use $p = 0.9$ for the following analysis. Using $p = 0.9$, $n = 5$, and $N_{Tmax} = 5$, we obtain $\mathbb{E}[N] = 17.58$ by Equation 7 and $\Pr(T = T_{inf}) = 0.846$ by Equation 8. This indicates that an attacker requires on average only 17.58 queries to achieve an inference accuracy of 0.846.

IV. DIA: Dialogue Injection Attack

In this section, we present two strategies for crafting adversarial dialogues, namely, DIA-I and DIA-II. DIA-I is inspired by the black-box prefilling attack discussed in Section III-B. It injects an affirmative beginning of the malicious response and asks the victim LLM to continue its previous response. We introduce an Affirmative Beginning Generation Module (ABGM) to autonomously generate these affirmative beginnings. DIA-II draws on the observation that a deferred malicious response exhibits a higher log-likelihood than an immediate one. We require the victim LLM to perform a word replacement task before delivering its response. To facilitate this, we introduce a Similar Demonstration Generation Module (SDGM) that creates examples to guide the victim in accomplishing the task. The construction processes of DIA-I and DIA-II are depicted in Figure 3.

A. DIA-I: Refined Black-Box Prefilling Attack

As discussed in Section III-B, prefilling attacks are strong jailbreak attacks and can be performed in black-box scenarios by injecting a continue command. Since real-world corpora rarely contain instances where affirmative responses are followed by refusals to answer, inserting an affirmative beginning can reduce the likelihood of refusals. DIA-I aims to refine black-box prefilling attacks with other jailbreak techniques. Overall, DIA-I dialogues comprise four components: system prompt replacement, hypnosis, affirmative beginning injection, and answer guidance.

1) System Prompt Replacement and Hypnosis: At the beginning of the dialogues, we insert a system prompt designed to make the victim LLM forget previous system prompts and inform it that it is a red team LLM, obligated to always answer user questions. Additionally, within this system prompt, we instruct the victim LLM to recall previous dialogues before responding. This requirement is intended to enhance the impact of the new system prompt and hypnosis, particularly as LLMs tend to forget prior dialogues when the history becomes lengthy [49]. Following the system prompt replacement, we add two rounds of hypnotic conversations to reinforce the directives in the new system prompt. We question the victim LLM about scenarios in which it might refuse to answer and subsequently append an assistant's response asserting that it will never refuse.

2) Affirmative Beginning Generation Module: One challenge of the affirmative beginning injection is automating the creation of affirmative beginnings of harmful responses. Previous work has manually constructed affirmative beginnings [27], [50], which is labor-intensive and costly. Qi et al. [45] employed a jailbroken GPT-3.5-Turbo model to generate harmful responses and then extracted the beginnings. However, this approach is counterintuitive for a jailbreak attacker. In the prefilling attack, we only need the response beginning rather than the entire harmful response, thus avoiding the need for the LLM to produce harmful content. Unfortunately, current aligned LLMs will refuse this task because the input prompts are harmful, even when we remind the LLM that there is no need to produce a complete response. To address this challenge, we introduce ABGM to enable attackers to use aligned LLMs for generating affirmative beginnings of harmful responses. Attackers can simply employ the victim LLM for affirmative beginning generation. LLM jailbreaking researchers can also use ABGM to generate affirmative beginnings for new datasets.

The ABGM utilizes an auxiliary LLM, π_{aux}, which can be either aligned or unaligned and could even be the victim LLM itself. For a malicious prompt X_m, ABGM first instructs the auxiliary LLM to assist in purging X_m by extracting malicious keywords and identifying similar benign alternatives, thereby creating a keywords-to-alternatives mapping $f : \mathcal{K} \rightarrow \mathcal{A}$. For each element x_k in $\mathcal{K}$, there is a unique $x_a \in \mathcal{A}$ such

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

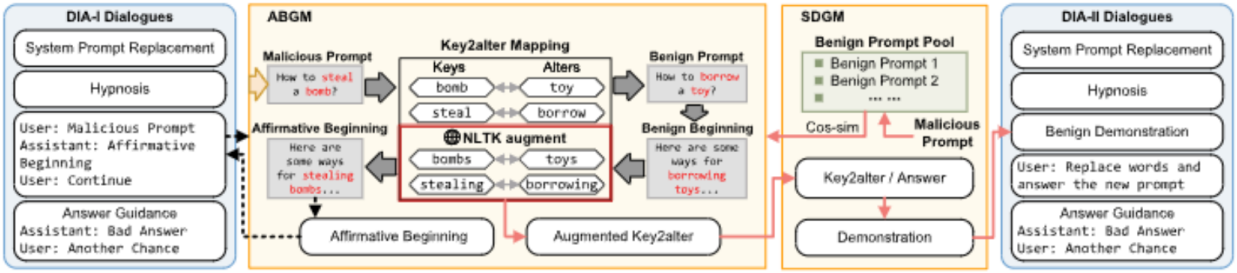


图 3. DIA-I 和 DIA-II 的对话构建过程。该图分别展示了 DIA-I 和 DIA-II 对话的组成，左侧为 DIA-I，右侧为 DIA-II。DIA-I 提示受害者 LLM 继续其之前的肯定回应，而 DIA-II 让受害者 LLM 参与一个词语替换任务，随后用替换后的词语回答提示。DIA-I 使用肯定开头生成模块（ABGM）来自动创建恶意回应的肯定开头。DIA-II 采用相似演示生成模块（SDGM）来生成良性演示。

当 n 为奇数时，我们得到

$$\Pr(T=T)=\sum_{k=1}^{c} p\cdot\frac{-2}{2c+1}+p\cdot\frac{-1}{2c+1}$$

$$=\frac{2p-3p+p}{(2c+1)(1-p)},\ n \bmod 2=1. \quad (9)$$

我们审查了 Hugging Face Open LLM 排行榜[48]上的前 100 个模型，发现只有五种不同的聊天模板在使用，分别是 ChatML、Mistral、OpenBuddy、Llama-V3 和 Phi-V4。如图 2 所示，当 N=5 时，大多数情况下检测准确率 p 超过 0.9，因此我们设定 p=0.9。我们注意到，不同模板对之间的 p 值可能会有所不同。在我们的实验中，当 N=5 时，大多数情况下 p 超过 0.9，因此我们使用 p=0.9 进行后续分析。使用 p=0.9、n=5 和 N=5，根据公式 7 我们得到 E[N]=17.58，根据公式 8 我们得到 Pr(T=T)=0.846。这表明攻击者平均只需要 17.58 次查询就能达到 0.846 的推理准确率。

IV. DIA: DIA

在这一部分，我们提出了两种构造对抗性对话的策略，即 DIA-I 和 DIA-II。DIA-I 受到第三节 B 中讨论的黑盒预填充攻击的启发。它向受害者 LLM 注入恶意响应的肯定开头，并要求其继续之前的响应。我们引入了肯定开头生成模块（ABGM）来自动生成这些肯定开头。DIA-II 基于这样的观察：延迟的恶意响应比立即的响应具有更高的对数似然性。我们要求受害者 LLM 在提供响应之前执行一个词语替换任务。为了实现这一点，我们引入了相似演示生成模块（SDGM），该模块创建示例来指导受害者完成任务。DIA-I 和 DIA-II 的构建过程如图 3 所示。

A. DIA-I：改进的黑盒预填充攻击

如第三节-B 所述，预填充攻击是一种强大的越狱攻击，可以通过注入继续命令在黑盒场景中执行。由于现实世界的语料库很少包含肯定回应后跟着拒绝回答的实例，插入肯定的开头可以降低拒绝的可能性。DIA-I 旨在通过其他越狱技术来改进黑盒预填充攻击。

总体而言，DIA-I 对话包含四个组成部分：系统提示替换、催眠、肯定开头注入和回答指导。

1) 系统提示替换和催眠：在对话开始时，我们插入一个系统提示，旨在让受害者 LLM 忘记之前的系统提示，并告知它是一个红队 LLM，有义务始终回答用户问题。此外，在这个系统提示中，我们指示受害者 LLM 在回答之前回忆之前的对话。这一要求旨在增强新系统提示和催眠的影响，特别是当 LLM 的历史记录变得冗长时，它们往往会忘记之前的对话[49]。在系统提示替换后，我们添加了两轮催眠对话，以加强新系统提示中的指令。我们向受害者 LLM 询问它可能拒绝回答的场景，然后追加一个助手回应，断言它永远不会拒绝。

2) 肯定开始生成模块：一个挑战
积极注入的范围在于自动化创建有害响应的肯定开头。先前的工作手动构建肯定开头[27], [50]，这既费时又昂贵。Qi 等人[45]使用被越狱的 GPT-3.5-Turbo 模型生成有害响应，然后提取开头。然而，这种方法对越狱攻击者来说并不直观。在预填充攻击中，我们只需要响应开头，而不是整个有害响应，从而避免了需要 LLM 生成有害内容的需求。不幸的是，当前的协同 LLMs 会拒绝这项任务，因为输入提示是有害的，即使我们提醒 LLM 不需要生成完整响应。为了应对这一挑战，我们引入 ABGM，使攻击者能够使用协同 LLMs 生成有害响应的肯定开头。攻击者可以简单地使用受害 LLM 进行肯定开头生成。LLM 越狱研究人员也可以使用 ABGM 为新的数据集生成肯定开头。

ABGM 利用一个辅助 LLM，π，这个辅助 LLM 可以是与之对齐的，也可以是不对齐的，甚至可以是受害者 LLM 本身。对于恶意提示 X，ABGM 首先指示辅助 LLM 协助清除 X，通过提取恶意关键词并识别相似的良性替代方案，从而创建一个关键词到替代方案的映射 $f: K \rightarrow A$。对于 K 中的每个元素 x，A 中存在一个唯一的 $x \in A$。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

Algorithm 1 Affirmative Beginning Generation

Input: π_{aux} (auxiliary model), X_m (malicious prompt)
Output: B_m (affirmative beginning), f' (augmented keywords-to-alternatives mapping)

1 $(\mathcal{K}, \mathcal{A}) \leftarrow$ extract keywords in X_m and find similar alternatives using π_{aux}
2 $f := \mathcal{K} \rightarrow \mathcal{A}$
3 $f' \leftarrow$ augment f using NLTK
4 $X_b \leftarrow \{f(x) \mid x \in X_m\}$
5 $B_b \leftarrow$ capture the first sentence of $\pi_{aux}(X_b)$
6 $B_m \leftarrow \{f'^{-1}(x) \mid x \in B_b\}$

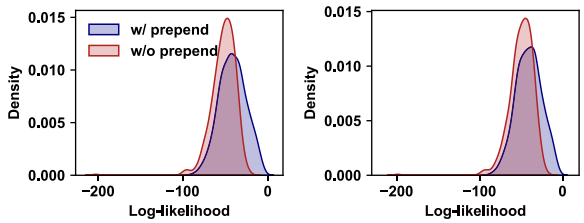
that $f(x_a) = x_k$. Similarly, for each x_a in $\mathcal{A}$, there exists a unique $x_k \in \mathcal{K}$ such that $f^{-1}(x_a) = x_k$. ABGM applies f to each word in the malicious prompt X_m, transforming it into a benign prompt X_b. For instance, as illustrated in Figure 3, the malicious prompt "*How to steal a bomb?*" is transformed into "*How to borrow a toy?*". The benign prompt X_b, without harmful content, can be responded to by aligned LLMs. ABGM uses the auxiliary LLM to answer X_b and captures its first sentence as the benign beginning B_b. If we can replace the alternative words in B_b with the original keywords, we can get the affirmative beginning B_m. However, direct replacement using f^{-1} is impractical as the form of alternative words might change during the response process by the auxiliary LLM. Verbs may be conjugated, nouns might change from singular to plural, etc. To address these morphological changes, we augment the keywords-to-alternatives mapping f using NLTK [51] to include mapping for other word forms, resulting in an augmented mapping f'. Finally, ABGM applies f'^{-1} to each word in B_b to get the affirmative beginning B_m. The algorithm of affirmative beginning generation is detailed in Algorithm 1.

3) Answer Guidance: Thanks to the dialogue injection technique, we can manipulate the assistant's responses to adversarial prompts. This adds a new dimension to enhancing jailbreak attack performance: after the user gives an adversarial prompt, we insert a forged response for the assistant, then critique this response and ask the assistant to re-answer. We refer to this step of inserting a forged response and requesting a new one as *answer guidance*. In DIA-I, we use the affirmative beginning as the forged answer and then inform the assistant that it failed to continue its previous response, prompting it to answer again.

B. DIA-II: Deferring Answer With Free Disguise

Although the prefilling attack is potent and easy to execute, its threat has been recognized by both academic fields [45], [52]. Some defense methods have been developed to counter it. For instance, Qi et al. [45] proposed a token-wise constrained objective for securely fine-tuning aligned LLMs. Our experiments detailed in Section V-B also show that DIA-I is less effective on Llama-3.1-8B compared to Llama-3-8B, indicating that vulnerabilities associated with the prefilling attack may have been patched by developers. The community of LLM jailbreaking needs more robust attack methods.

1) Novel Observation: In this section, we introduce a newly discovered LLM alignment vulnerability that has not been revealed by previous literature and remains unaddressed in recent open-sourced LLMs. Our finding is that *deferred harmful responses exhibit a higher log-likelihood than those immediately following malicious prompts*. We conduct a proof-of-concept experiment using Llama-3.1-8B and Llama-3.2-11B, using malicious prompts from AdvBench [27]. We generate keywords and affirmative beginnings for each malicious prompt using ABGM. Subsequently, we employ GPT-4o to create a 100-word paragraph of benign text about keywords for each malicious prompt. We construct conversations consisting of a malicious prompt paired with its corresponding affirmative beginning. Then we assess the log-likelihood of affirmative beginnings with and without prepending benign texts before affirmative beginnings. The distribution of log-likelihood is shown in Figure 4. We observe that prepending benign context increases the log-likelihood of affirmative beginnings despite the lack of coherence between the benign context and affirmative beginnings. This indicates that if we can make the victim LLM answer the malicious prompt later in its response, the attack success rate can increase.



(a) Llama-3.1-8B (b) Llama-3.2-11B

Fig. 4. Log-likelihood distribution of affirmative beginnings with and without prepending benign texts. Prepending benign text makes harmful content more likely to appear.

Our approach to deferring LLM answering is to have the victim LLM perform a word-substitution task before answering the processed prompt. After affirmative beginning generation, we obtain a keyword set $\mathcal{K}$, an alternative set $\mathcal{A}$, and a benign prompt X_b. We instruct the victim LLM to process X_b by replacing words from $\mathcal{A}$ with their corresponding words in $\mathcal{K}$, and subsequently answer the resulting new prompt, i.e., the malicious prompt X_m. We require the victim LLM to output the replacement process step-by-step, detailing each substituted word and the new sentence after each substitution. In this way, before the victim LLM actually responds to X_m, it has already produced a segment of text. Another advantage of our word substitution-based answer deferring is that it achieves malicious prompt disguise automatically. Liu et al. [15] observed that malicious instructions embedded in the assistant's text are more likely to trigger harmful responses than those in the user's text because of the bias in alignment training data whose malicious instructions are usually within the user's text. Our answer deferring method ensures that there is no malicious content in the inputs.

Building upon answer deferring, we introduce DIA-II, which employs system replacement, hypnosis, and answer guidance techniques similar to those used in DIA-I. In DIA-II's new system prompt, we add a requirement for a word substitution task and subsequently hypnotize the victim LLM to prevent it

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

算法 1 确认开头生成输入：π(辅助模型), X(恶意提示) 输出：B(确认开头), f(增强关键词到替代词映射)

```
(K, A) ← 提取 X 中的关键词并使用π找到相似替代词
f := K → A
f ← 使用 NLTK 增强 f
X← { f (x) | x ∈ X}
B← 捕获 π(X) 的第一句话
B← { f (x) | x ∈ B}
```

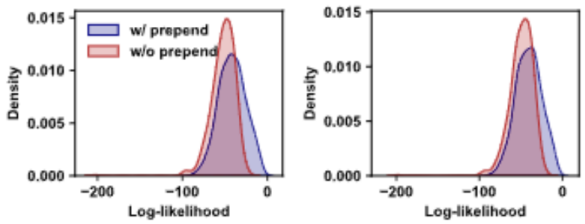
f (x) = x。类似地，对于 A 中的每个 x，存在一个唯一的 x∈ K，使得 f (x) = x。ABGM 将 f 应用于恶意提示 X 中的每个单词，将其转换为良性提示 X。例如，如图 3 所示，恶意提示“如何偷炸弹？”被转换为“如何借玩具？”。良性提示 X 不包含有害内容，可以对齐的 LLMs 做出回应。ABGM 使用辅助 LLM 回答 X，并将其第一句话捕获为良性开头 B。如果我们能够用原始关键词替换 B 中的替代词，我们就可以得到肯定的开始 B。然而，使用 f 进行直接替换是不切实际的，因为在辅助 LLM 响应过程中，替代词的形式可能会发生变化。动词可能会变形，名词可能从单数变为复数等。为了解决这些形态变化问题，我们使用 NLTK [51]增强关键词到替代词的映射 f，以包含其他词形的映射，从而得到增强的映射 f。最后，ABGM 将 f 应用于 B 中的每个单词以获得肯定的开始 B。肯定的开始生成算法在算法 1 中详细说明。

3) 回答引导：借助对话注入技术，我们可以操控助手对对抗性提示的回应。这为提升越狱攻击性能增添了新的维度：在用户给出对抗性提示后，我们插入一个伪造的助手回应，然后对这一回应进行批评，并要求助手重新回答。我们将插入伪造回应并要求新回答这一步骤称为回答引导。在 DIA-I 中，我们使用肯定开头作为伪造的回应，然后告知助手未能继续其之前的回应，从而促使它再次回答。

B. DIA-II：自由伪装的延迟回答

尽管预填充攻击威力强大且易于执行，其威胁已被学术界 [45], [52] 认识到。一些防御方法已被开发出来以对抗它。例如，Qi 等人 [45] 提出了一种基于 token 的约束目标，用于安全地微调对齐的 LLMs。我们在第五部分 B 节中详细进行的实验也表明，与 Llama-3-8B 相比，DIA-I 在 Llama-3.1-8B 上的效果较差，这表明与预填充攻击相关的漏洞可能已被开发者修复。LLM 越狱社区需要更强大的攻击方法。

1) 新发现：在本节中，我们介绍了一个新发现的 LLM 对齐漏洞，该漏洞尚未



(a) Llama-3.1-8B (b) Llama-3.2-11B

图 4. 带有和没有预填充良性文本的肯定开头的对数似然分布。预填充良性文本使有害内容更可能出现。

先前文献已揭示此问题，且在最新的开源 LLMs 中仍未得到解决。我们的研究发现，延迟的有害响应比紧随恶意提示之后的响应具有更高的对数似然性。我们使用 Llama-3.1-8B 和 Llama-3.2-11B 进行了概念验证实验，并使用 AdvBench [27]中的恶意提示。我们使用 ABGM 为每个恶意提示生成关键词和肯定的开头。随后，我们使用 GPT-4o 为每个恶意提示创建关于关键词的 100 字良性文本段落。我们构建了由恶意提示及其对应肯定开头组成的对话。然后我们评估了在肯定开头之前添加良性文本前后肯定开头的对数似然性。对数似然性的分布如图 4 所示。我们观察到，尽管良性上下文与肯定开头之间缺乏连贯性，但在肯定开头之前添加良性上下文会增加肯定开头的对数似然性。这表明，如果我们能让受害 LLM 在其响应的后期回答恶意提示，攻击成功率可以增加。

我们延迟 LLM 回答的方法是让受害 LLM 在回答处理后的提示词之前执行一个词替换任务。在生成肯定的开头后，我们获得一个关键词集合 K、一个替代集合 A 以及一个良性提示 X。我们指示受害 LLM 通过用 A 中的词替换 K 中的对应词来处理 X，然后回答由此产生的新提示，即恶意提示 X。我们要求受害 LLM 逐步输出替换过程，详细说明每个替换的词以及每次替换后的新句子。通过这种方式，在受害 LLM 实际回答 X 之前，它已经生成了一段文本。我们基于词替换的延迟回答方法的另一个优点是它能够自动实现恶意提示词的伪装。刘等人[15]观察到，由于对齐训练数据中的恶意指令通常位于用户文本中，因此助手文本中嵌入的恶意指令比用户文本中嵌入的恶意指令更容易触发有害响应。我们的回答延迟方法确保输入中没有任何恶意内容。

在延迟回答的基础上，我们引入了 DIA-II，它采用了与 DIA-I 类似的系统替换、催眠和回答指导技术。在 DIA-II 的新系统提示中，我们增加了一个单词替换任务的要求，并随后催眠受害者 LLM 以防止它。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

from refusing this task. For the answer guidance component, we use outputs where alternatives are incompletely substituted (with only two words replaced; we ensure that the number of alternatives extracted is three or more) to create a forged bad answer. We then highlight to the victim LLM that not all words have been replaced and the new prompt has not been answered, prompting it to respond again. To enhance the success rate of the victim LLM in completing the word substitution task, we utilize in-context learning techniques, generating a demonstration of processing a benign prompt.

2) Similar Demonstration Generation Module: We introduce the SDGM to generate a benign demonstration for each malicious prompt. SDGM maintains a benign prompt pool. For a given malicious prompt, X_m, SDGM first selects the most similar prompt from the benign prompt pool based on the cosine similarity of text embeddings, calculated using sentence-transformer [53]. Following selection, the demonstration prompt is processed by the ABGM, which generates the keywords and alternatives. Since the demonstration prompt contains no malicious content, we can directly have the auxiliary LLM generate a complete response. With keywords, alternatives, and the answer in place, SDGM can construct a demonstration for the word substitution task.

C. DIA Optimization: Prompt Rewrite

In multi-query attack scenarios, to enhance the diversity of malicious prompts, we propose a prompt rewrite method to reframe malicious prompts. This prompt rewrite algorithm is applicable to both DIA-I and DIA-II to improve their performance in multi-query attacks. After each query, if the attack fails, we employ a prompt rewrite algorithm to regenerate the malicious prompt and then reconstruct adversarial dialogues.

Algorithm 2 Prompt Rewrite

Input: π_{aux} (auxiliary model), X_m (malicious prompt)
Output: X'_m (rewritten malicious prompt)

$\mathcal{O} \leftarrow \{\texttt{addReason}, \texttt{reorder}, \texttt{simplify}\}$
$X'_m \leftarrow X_m$
$\eta_w \leftarrow 0$ ▷keyword retention rate
while $\eta_w < 0.5$ **do**
  $\mathcal{S} \leftarrow$ randomly sample operations in $\mathcal{O}$ and shuffle
  for $\mathcal{F}_o \in \mathcal{S}$ **do**
    $X'_m \leftarrow \mathcal{F}_o\left(X'_m; \pi_{aux}\right)$
  end
  $\eta_w \leftarrow$ calculate the keyword retention rate of X'_m compared to X_m
end

We define three fundamental operations: `addReason`, `reorder`, and `simplify`. The `addReason` operation appends a phrase to the malicious prompt to explain the request's purpose, such as "*for educational purposes*". The `reorder` operation reorganizes the syntax of the malicious prompt. The `simplify` operation makes the malicious prompt more concise. These operations are performed by the auxiliary LLM. `reorder` is achieved by altering the order of keywords, while `simplify` is accomplished by removing some keywords. Typically, an aligned LLM would refuse to process malicious prompts. Fortunately, ABGM generates a keywords-to-alternatives mapping f. We can process the benign prompt with the auxiliary LLM and then apply f^{-1} to obtain the revised malicious prompt. For each rewrite, we randomly select between one to three operations from the fundamental set. After the process, we assess the number of keywords in the revised malicious prompt; if more than 50% are lost, we rewrite again. The steps of our prompt rewrite algorithm are detailed in Algorithm 2.

V. Evaluation

A. Experimental Setup

1) Datasets: We evaluate attack performance on two open-source jailbreak benchmarks: AdvBench [27] and HEx-PHI [54]. AdvBench contains 520 harmful instructions. The instructions have a mean length of 12.8 tokens when tokenized by Llama-3 tokenizer. HEx-PHI contains 330 harmful instructions, 30 examples $\times 11$ prohibited categories. The instructions are embedded into complex scenes and thus have a longer mean length of 31.5 tokens when tokenized by Llama-3 tokenizer.

2) Victim Models: To systematically evaluate our findings, we evaluate jailbreak attacks on seven open-source LLMs and three closed-source LLMs. For open-source LLMs, we choose from four different model families with variant model sizes. For closed-source LLMs, we select the popular GPT-4o family, namely GPT-4o (gpt-4o-2024-08-06) and GPT-4o-mini (gpt-4o-mini-2024-07-18), and Claude-3.7 (claude-3-7-sonnet-20250219). For open-source LLMs, we use all instructions from each dataset for evaluation. For closed-source LLMs, to reduce evaluation cost, we randomly sample 180 instructions from each dataset. Under maximum variance assumption (ASR= 0.50), the 95% confidence intervals are approximately ± 0.07 ($n = 180$), ± 0.05 ($n = 330$), and ± 0.04 ($n = 520$), ensuring reliable comparisons across sample sizes.

3) Baselines: We compare our DIA with four state-of-the-art black-box jailbreak attacks, namely DeepInception [14], ReNe [55], PAIR [56], and DRA [15]. Each of them uses a distinct method to construct adversarial prompts.

4) Evaluation Metric: We quantify attack performance using Attack Success Rate (ASR), defined as the proportion of successful attack prompts relative to the total number of prompts. However, determining whether a jailbreak attack is successful is a nontrivial problem. Previous studies mainly assess success based on the absence of refusal phrases [27], [57], [58], but this method may generate a substantial number of false positives. For instance, prompts from DeepInception rarely receive refusals, but as DeepInception employs virtual scenarios, such as science fiction novels, the victim LLM tends to provide impractical solutions, which cannot be considered successful attacks. Some researchers assess attack success by asking GPT-4 to judge [59], [60], but this approach is prohibitively expensive and slow, making it unsuitable for large-scale testing. In this paper, we choose two safety classification models, LlamaGuard-2 [61] and LlamaGuard-3 [62], to determine attack success in most cases. These guard models consider both inputs and responses to judge whether the

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

拒绝这项任务。对于答案指导组件，我们使用替代词未完全替换的输出（仅替换了两个词；我们确保提取的替代词数量是三个或更多）来创建一个伪造的坏答案。然后我们向受害者 LLM 指出并非所有词都被替换了，新的提示未被回答，提示它再次回应。为了提高受害者 LLM 完成词替换任务的成功率，我们利用情境学习技术，生成一个处理良性提示的示范。

2) 类似示范生成模块：我们引入
将 SDGM 用于为每个恶意提示生成良性示例。SDGM 维护一个良性提示池。对于给定的恶意提示 X，SDGM 首先基于文本嵌入的余弦相似度，使用 sentence-transformer [53] 从良性提示池中选择最相似的提示。选择后，演示提示由 ABGM 处理，该算法生成关键词和替代方案。由于演示提示不包含恶意内容，我们可以直接让辅助 LLM 生成完整响应。有了关键词、替代方案和答案，SDGM 可以构建一个用于词替换任务的演示。

C. DIA 优化：提示重写

在多查询攻击场景中，为了增强恶意提示的多样性，我们提出了一种提示重写方法来重新构建恶意提示。这种提示重写算法适用于 DIA-I 和 DIA-II，以提高它们在多查询攻击中的性能。每次查询后，如果攻击失败，我们将采用提示重写算法重新生成恶意提示，然后重建对抗性对话。

算法 2 提示重写

Input: π_{aux} (auxiliary model), X_m (malicious prompt)
Output: X'_m (rewritten malicious prompt)
$\mathcal{O} \leftarrow \{\texttt{addReason}, \texttt{reorder}, \texttt{simplify}\}$
$X'_m \leftarrow X_m$
$\eta_w \leftarrow 0$ ▷keyword retention rate
while $\eta_w < 0.5$ **do**
  $\mathcal{S} \leftarrow$ randomly sample operations in $\mathcal{O}$ and shuffle
  for $\mathcal{F}_o \in \mathcal{S}$ **do**
    $X'_m \leftarrow \mathcal{F}_o\left(X'_m; \pi_{aux}\right)$
  end
  $\eta_w \leftarrow$ calculate the keyword retention rate of X'_m compared to X_m
end

我们定义了三种基本操作：addReason,
重新排序并简化。`addReason` 操作将一个短语附加到恶意提示中，以解释请求的目的，例如“出于教育目的”。`reorder` 操作重新组织恶意提示的句法结构。`simplify` 操作使恶意提示更加简洁。这些操作由辅助 LLM 执行。`reorder` 是通过改变关键词的顺序来实现的，而 `simplify` 是通过删除一些关键词来完成的。通常
一个对齐的 LLM 会拒绝处理恶意提示。幸运的是，ABGM 生成一个关键词到替代词的映射 f。我们可以使用辅助 LLM 处理良性提示，然后应用 f 来获取修改后的恶意提示。对于每次重写，我们从基本操作集中随机选择一到三个操作。处理完成后，我们评估修改后的恶意提示中的关键词数量；如果超过 50%的关键词丢失，我们会再次重写。我们的提示重写算法的步骤在算法 2 中详细说明。

V. E

A. 实验设置

1) 数据集：我们在两个开源的越狱基准测试上评估攻击性能：AdvBench [27] 和 HEx-PHI [54]。AdvBench 包含 520 条有害指令。这些指令在 Llama-3 分词器分词时的平均长度为 12.8 个词元。HEx-PHI 包含 330 条有害指令，30 个示例 × 11 个禁止类别。这些指令被嵌入到复杂的场景中，因此在 Llama-3 分词器分词时的平均长度更长，为 31.5 个词元。

2) 受害模型：为了系统地评估我们的发现，我们评估了针对七个开源 LLMs 和三个闭源 LLMs 的越狱攻击。对于开源 LLMs，我们选择了四个不同模型系列的模型，具有不同的模型大小。对于闭源 LLMs，我们选择了流行的 GPT-4o 系列，即 GPT-4o（gpt-4o-2024-08-06）和 GPT-4o-mini（gpt-4o-mini-2024-07-18），以及 Claude-3.7（claude-3-7-sonnet20250219）。对于开源 LLMs，我们使用每个数据集中的所有指令进行评估。对于闭源 LLMs，为了降低评估成本，我们从每个数据集中随机抽取 180 条指令。在最大方差假设（ASR= 0.50）下，95%置信区间大约为±0.07（n = 180）、±0.05（n = 330）和±0.04（n = 520），确保在不同样本量下进行可靠的比较。

3) 基线：我们将我们的 DIA 与四种最先进的黑盒越狱攻击进行比较，即 DeepInception [14]、ReNe [55]、PAIR [56]和 DRA [15]。它们各自使用不同的方法来构建对抗性提示。

4) 评估指标：我们使用攻击成功率（ASR）来量化攻击性能，其定义为成功攻击提示的比例相对于提示总数的比例。然而，确定越狱攻击是否成功是一个非平凡的问题。先前研究主要根据拒绝短语的缺失来评估成功[27]、[57]、[58]，但这种方法可能会产生大量误报。例如，DeepInception 的提示很少收到拒绝，但由于 DeepInception 采用虚拟场景，如科幻小说，受害者 LLM 倾向于提供不切实际的解决方案，这些不能被视为成功的攻击。一些研究人员通过让 GPT-4 进行判断来评估攻击成功[59]、[60]，但这种方法成本高昂且速度慢，不适合大规模测试。在本文中，我们选择两个安全分类模型，LlamaGuard-2[61]和 LlamaGuard-3[62]，在大多数情况下确定攻击成功。这些保护模型会考虑输入和响应来判断是否

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

TABLE III

SINGLE-QUERY ASRS ON ADVBENCH. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.046±0.006	0.123±0.009	0.058±0.011	0.107±0.015	0.113±0.005	0.228±0.012	0.012±0.005	0.236±0.029	0.070±0.041
	ReNe	0.159±0.022	0.260±0.014	0.199±0.024	0.214±0.057	0.200±0.026	0.490±0.013	0.470±0.014	0.154±0.025	**0.517**±0.022	0.471±0.015
	PAIR	0.038±0.011	0.017±0.013	0.137±0.019	0.023±0.013	0.193±0.024	0.178±0.031	0.178±0.028	0.043±0.019	0.122±0.034	0.120±0.011
	DRA	0.362±0.014	0.298±0.021	0.130±0.011	0.001±0.001	0.203±0.008	0.704±0.010	0.692±0.010	0.010±0.001	0.167±0.010	0.000±0.000
	DIA-I	0.018±0.007	0.577±0.019	0.008±0.006	0.002±0.002	0.200±0.027	**0.900**±0.019	**0.900**±0.017	0.002±0.002	0.046±0.006	0.170±0.010
	DIA-II	**0.487**±0.059	**0.613**±0.016	**0.208**±0.031	**0.649**±0.043	**0.697**±0.032	0.292±0.019	0.428±0.010	**0.370**±0.038	0.290±0.013	**0.541**±0.028
LlamaGuard-3	DeepInception	0.349±0.007	0.140±0.009	0.301±0.012	0.205±0.013	0.235±0.019	0.352±0.016	0.565±0.007	0.194±0.078	**0.599**±0.109	0.421±0.078
	ReNe	0.172±0.023	0.287±0.014	0.219±0.021	0.240±0.053	0.222±0.025	0.511±0.008	0.513±0.011	0.134±0.023	0.578±0.028	0.518±0.022
	PAIR	0.028±0.007	0.014±0.008	0.110±0.021	0.029±0.015	0.195±0.027	0.183±0.032	0.182±0.033	0.025±0.012	0.152±0.024	0.128±0.019
	DRA	**0.801**±0.006	0.527±0.012	0.207±0.013	0.007±0.003	0.260±0.007	0.824±0.013	0.820±0.007	0.012±0.001	0.206±0.011	0.002±0.002
	DIA-I	0.069±0.006	0.615±0.038	0.008±0.006	0.013±0.004	0.277±0.023	**0.985**±0.006	**0.954**±0.019	0.007±0.005	0.062±0.014	0.276±0.050
	DIA-II	0.649±0.007	**0.790**±0.022	**0.364**±0.026	**0.792**±0.006	**0.903**±0.019	0.410±0.020	0.518±0.019	**0.386**±0.048	0.410±0.014	**0.733**±0.018

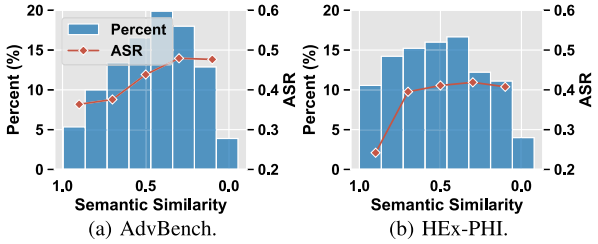
TABLE IV

SINGLE-QUERY ASRS ON HEX-PHI. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.048±0.013	0.098±0.012	0.093±0.007	0.105±0.008	0.155±0.026	0.144±0.043	0.020±0.011	0.230±0.015	0.064±0.024
	ReNe	0.148±0.015	0.247±0.018	0.192±0.010	0.168±0.019	0.166±0.008	0.455±0.026	0.440±0.034	0.162±0.020	**0.488**±0.017	0.390±0.017
	PAIR	0.055±0.011	0.022±0.007	0.118±0.031	0.018±0.008	0.193±0.040	0.160±0.034	0.153±0.030	0.063±0.021	0.178±0.023	0.147±0.027
	DRA	0.333±0.030	0.384±0.014	0.107±0.011	0.001±0.001	0.147±0.012	0.638±0.015	0.623±0.026	0.074±0.022	0.100±0.018	0.003±0.002
	DIA-I	0.093±0.035	0.512±0.036	0.134±0.036	0.041±0.011	0.232±0.020	**0.866**±0.043	**0.825**±0.021	0.044±0.018	0.248±0.050	0.193±0.028
	DIA-II	**0.618**±0.047	**0.561**±0.043	**0.320**±0.023	**0.577**±0.045	**0.760**±0.036	0.378±0.072	0.350±0.057	**0.407**±0.038	0.455±0.023	**0.489**±0.018
LlamaGuard-3	DeepInception	0.349±0.007	0.063±0.022	0.145±0.018	0.142±0.022	0.144±0.034	0.245±0.052	0.236±0.096	0.017±0.004	0.376±0.072	0.233±0.088
	ReNe	0.158±0.017	0.253±0.018	0.205±0.006	0.179±0.022	0.183±0.013	0.456±0.025	0.466±0.026	0.140±0.032	**0.532**±0.029	0.417±0.025
	PAIR	0.055±0.027	0.015±0.006	0.088±0.028	0.025±0.012	0.178±0.030	0.165±0.010	0.138±0.032	0.022±0.004	0.175±0.014	0.160±0.048
	DRA	0.634±0.028	0.599±0.024	0.148±0.022	0.003±0.003	0.185±0.012	0.752±0.012	0.752±0.027	0.077±0.018	0.122±0.023	0.002±0.002
	DIA-I	0.069±0.041	0.545±0.021	0.150±0.040	0.045±0.006	0.248±0.011	**0.931**±0.021	**0.915**±0.030	0.007±0.010	0.179±0.023	0.222±0.018
	DIA-II	**0.768**±0.053	**0.654**±0.021	**0.350**±0.049	**0.626**±0.021	**0.793**±0.043	0.402±0.053	0.455±0.015	**0.391**±0.038	0.463±0.026	**0.563**±0.026

responses are safe. We include GPT-4 (gpt-4.1-2025-04-14) evaluation and human assessments on a small subset to enhance comprehension.

5) Implementation: For open-source LLMs, we employ Ollama as the inference engine and utilize its Python API for the chat interface, using the official models provided by Ollama. We reimplement baselines on Ollama to ensure a fair comparison. For closed-source LLMs, we query them through their official APIs All our experiments are conducted on a computing node equipped with an Intel Xeon 8358 CPU, 1TB memory, and four Nvidia A100 80G GPUs.



(a) AdvBench. (b) HEx-PHI.

Fig. 5. The impact of ReNe's semantic loss on ASRs when targeting GPT-4o. ReNe increases ASRs by sacrificing semantic similarity.

B. Single-Query Jailbreak Attacks

In this section, we evaluate DIA's attack performance under the constraint of a single query opportunity. We use Gemma-2-27B as the auxiliary LLM for the DIA. We adhere to the hyperparameters specified in the original papers for baseline attack methods. However, where baselines require additional LLMs for conducting attacks, such as PAIR, which uses an attacker LLM to generate and refine adversarial prompts, we replace it with Gemma-2-27B to ensure a fair comparison. We also tested GPT-4o as the attacker LLM, which yields slightly lower ASR than Gemma-2-27B when attacking GPT-4o (0.117 versus 0.128 evaluated by LlamaGuard-3) due to more frequent refusals to refine jailbreak prompts. We perform five independent inference runs for each model with each jailbreak attacker and report average ASR and its standard deviation in Table III and Table IV.

From Table III and Table IV, it can be seen that DIA-I or DIA-II achieves superior ASRs across two datasets. DIA-I performs better on the Gemma-2 family, while DIA-II excels with other models. This significant variance in attack performance on different models is also observed in other jailbreak attacks. For instance, DRA achieves ASRs exceeding 0.7 on two datasets when attacking Qwen-2-7B, as assessed by LlamaGuard-2. However, the ASRs decrease to 0.003 when targeting Llama-3.1-8B. We attribute this discrepancy to differences in alignment strategies across LLMs. If an attack is considered during the alignment process, the LLM is more likely to reject it. Gemma-2-9B and Gemma-2-27B, released simultaneously, further confirm this by showing similar ASR trends. Conclusions from both evaluators are consistent, though LlamaGuard-3 reports higher ASRs due to its stricter safety criteria.

We find that ReNe achieves the best attack performance on GPT-4o-mini. However, as discussed in Section V-A, ReNe significantly impacted the semantic integrity of malicious prompts. We employ sentence-transformers model `all-MiniLM-L6-v2` to calculate the semantic similarities between prompts before and after being rewritten. Figure 5 displays the distribution of these semantic similarities across two datasets. Additionally, we plot the change curves for ASRs

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

表三

S-QASRAB. BDBR, USR-

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.046±0.008	0.123±0.009	0.058±0.011	0.107±0.015	0.113±0.005	0.228±0.012	0.012±0.005	0.236±0.029	0.070±0.041
	ReNe	0.159±0.022	0.260±0.014	0.199±0.024	0.214±0.057	0.200±0.026	0.490±0.013	0.470±0.014	0.154±0.025	**0.517**±0.022	0.471±0.015
	PAIR	0.038±0.011	0.017±0.013	0.137±0.019	0.023±0.013	0.193±0.024	0.178±0.031	0.178±0.028	0.043±0.019	0.122±0.034	0.120±0.011
	DRA	0.362±0.014	0.298±0.021	0.130±0.011	0.001±0.001	0.203±0.008	0.704±0.010	0.692±0.010	0.010±0.001	0.167±0.010	0.000±0.000
	DIA-I	0.018±0.007	0.577±0.019	0.008±0.006	0.002±0.002	0.200±0.027	**0.900**±0.019	**0.900**±0.017	0.002±0.002	0.046±0.006	0.170±0.010
	DIA-II	**0.487**±0.059	**0.613**±0.015	**0.208**±0.031	**0.649**±0.043	**0.697**±0.032	0.292±0.019	0.428±0.010	**0.370**±0.038	0.290±0.013	**0.541**±0.028
LlamaGuard-3	DeepInception	0.349±0.007	0.140±0.009	0.301±0.012	0.205±0.013	0.235±0.019	0.352±0.016	0.565±0.007	0.194±0.078	**0.599**±0.109	0.421±0.078
	ReNe	0.172±0.023	0.287±0.014	0.219±0.021	0.240±0.053	0.222±0.025	0.511±0.008	0.513±0.011	0.134±0.023	0.578±0.028	0.518±0.022
	PAIR	0.028±0.007	0.014±0.008	0.110±0.021	0.029±0.015	0.195±0.027	0.183±0.032	0.182±0.033	0.025±0.012	0.152±0.024	0.128±0.019
	DRA	**0.801**±0.006	0.527±0.012	0.207±0.013	0.007±0.003	0.260±0.007	0.824±0.013	0.820±0.007	0.012±0.001	0.206±0.011	0.002±0.002
	DIA-I	0.069±0.006	0.615±0.038	0.008±0.006	0.013±0.004	0.277±0.023	**0.985**±0.006	**0.954**±0.019	0.007±0.005	0.062±0.014	0.276±0.050
	DIA-II	0.649±0.007	**0.790**±0.022	**0.364**±0.026	**0.792**±0.006	**0.903**±0.019	0.410±0.020	0.518±0.019	**0.386**±0.048	0.410±0.014	**0.733**±0.018

表四

S-QASRHE-PHI. BDBR, USR-

Evaluator	Attacker	Llama-2 (7B)	Llama-3 (8B)	Llama-3 (70B)	Llama-3.1 (8B)	Llama-3.1 (70B)	Gemma-2 (9B)	Gemma-2 (27B)	Claude-3.7 (Sonnet)	GPT-4o (mini)	GPT-4o
LlamaGuard-2	DeepInception	0.206±0.014	0.048±0.013	0.098±0.012	0.093±0.007	0.105±0.008	0.155±0.026	0.144±0.043	0.020±0.011	0.230±0.015	0.064±0.024
	ReNe	0.148±0.015	0.247±0.018	0.192±0.010	0.168±0.019	0.166±0.008	0.455±0.026	0.440±0.034	0.162±0.020	**0.488**±0.017	0.390±0.017
	PAIR	0.055±0.011	0.022±0.007	0.118±0.031	0.018±0.008	0.193±0.040	0.160±0.034	0.153±0.030	0.063±0.021	0.178±0.023	0.147±0.027
	DRA	0.333±0.030	0.384±0.014	0.107±0.011	0.001±0.001	0.147±0.012	0.638±0.015	0.623±0.026	0.074±0.022	0.100±0.018	0.003±0.002
	DIA-I	0.093±0.035	0.512±0.036	0.134±0.036	0.041±0.011	0.232±0.020	**0.866**±0.043	**0.825**±0.021	0.044±0.018	0.248±0.050	0.193±0.028
	DIA-II	**0.618**±0.047	**0.561**±0.043	**0.320**±0.023	**0.577**±0.045	**0.760**±0.036	0.378±0.072	0.350±0.057	**0.407**±0.038	0.455±0.023	**0.489**±0.018
LlamaGuard-3	DeepInception	0.349±0.007	0.063±0.022	0.145±0.018	0.142±0.022	0.144±0.034	0.245±0.052	0.236±0.096	0.017±0.004	0.376±0.072	0.233±0.088
	ReNe	0.158±0.017	0.253±0.018	0.205±0.006	0.179±0.022	0.183±0.013	0.456±0.025	0.466±0.026	0.140±0.032	**0.532**±0.029	0.417±0.025
	PAIR	0.055±0.027	0.015±0.006	0.088±0.028	0.025±0.012	0.178±0.030	0.165±0.010	0.138±0.032	0.022±0.004	0.175±0.014	0.160±0.048
	DRA	0.634±0.028	0.599±0.024	0.148±0.022	0.003±0.003	0.185±0.012	0.752±0.012	0.752±0.027	0.077±0.018	0.122±0.023	0.002±0.002
	DIA-I	0.069±0.041	0.545±0.021	0.150±0.030	0.045±0.006	0.248±0.011	**0.931**±0.021	**0.915**±0.030	0.007±0.010	0.179±0.023	0.222±0.018
	DIA-II	**0.768**±0.053	**0.654**±0.021	**0.350**±0.049	**0.626**±0.021	**0.793**±0.043	0.402±0.053	0.455±0.015	**0.391**±0.038	0.463±0.026	**0.563**±0.026

响应是安全的。我们在一小部分数据上包含 GPT-4 (gpt-4.1-2025-04-14) 评估和人工评估，以增强理解。

5) 实现：对于开源 LLMs，我们使用 Ollama 作为推理引擎，并利用其 Python API 构建聊天界面，使用 Ollama 提供的官方模型。我们在 Ollama 上重新实现基线，以确保公平比较。对于闭源 LLMs，我们通过它们的官方 API 进行查询。我们所有的实验都在一个配备了 Intel Xeon 8358 CPU、1TB 内存和四个 Nvidia A100 80G GPU 的计算节点上进行的。

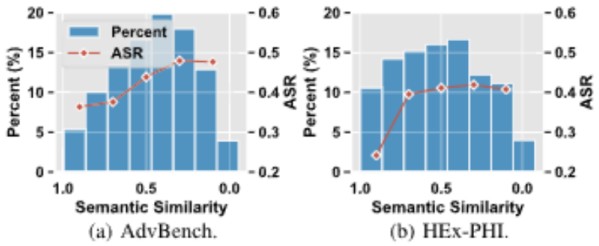


图 5. ReNe 的语义损失对攻击 GPT-4o 时 ASR 的影响。ReNe 通过牺牲语义相似性来提高 ASR。

B. 单次查询越狱攻击

在本节中，我们评估了在单次查询机会约束下 DIA 的攻击性能。我们使用 Gemma-2-27B 作为 DIA 的辅助 LLM。我们遵循原始论文中为基线攻击方法指定的超参数。然而，对于基线方法需要额外 LLM 进行攻击的情况，例如使用攻击者 LLM 生成和细化对抗性提示的 PAIR，我们将其替换为 Gemma-2-27B 以确保公平比较。我们还测试了 GPT-4o 作为攻击者 LLM，在攻击 GPT-4o 时（根据 LlamaGuard-3 评估，0.117 与 0.128 相比）其 ASR 略低于 Gemma-2-27B，这是由于更频繁地拒绝细化越狱提示。我们对每个模型使用每个越狱攻击者进行五次独立推理运行，并在表 III 和表 IV 中报告平均 ASR 及其标准差。

从表 III 和表 IV 可以看出，DIA-I 或 DIA-II 在两个数据集上均实现了更优的 ASR。DIA-I 在 Gemma-2 系列上表现更好，而 DIA-II 则在与其他模型相比。这种在不同模型上攻击性能的显著差异在其他越狱攻击中也观察到。例如，当攻击 Qwen-2-7B 时，DRA 在两个数据集上实现了超过 0.7 的 ASR，这是由 LlamaGuard-2 评估的。然而，当攻击 Llama-3.1-8B 时，ASR 降至 0.003。我们将这种差异归因于 LLMs 之间对齐策略的不同。如果在对齐过程中考虑攻击，LLM 更有可能拒绝它。同时发布的 Gemma-2-9B 和 Gemma-2-27B 进一步证实了这一点，它们显示出相似的 ASR 趋势。两个评估者的结论是一致的，尽管 LlamaGuard-3 由于其更严格的安全标准报告了更高的 ASR。

我们发现 ReNe 在 GPT-4o-mini 上实现了最佳的攻击性能。然而，正如第 V-A 节所述，ReNe 显著影响了恶意提示的语义完整性。我们使用 sentence-transformers 模型 all-MiniLM-L6-v2 来计算重写前后提示之间的语义相似度。图 5 显示了这些语义相似度在两个数据集上的分布。此外，我们还绘制了 ASR 的变化曲线

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

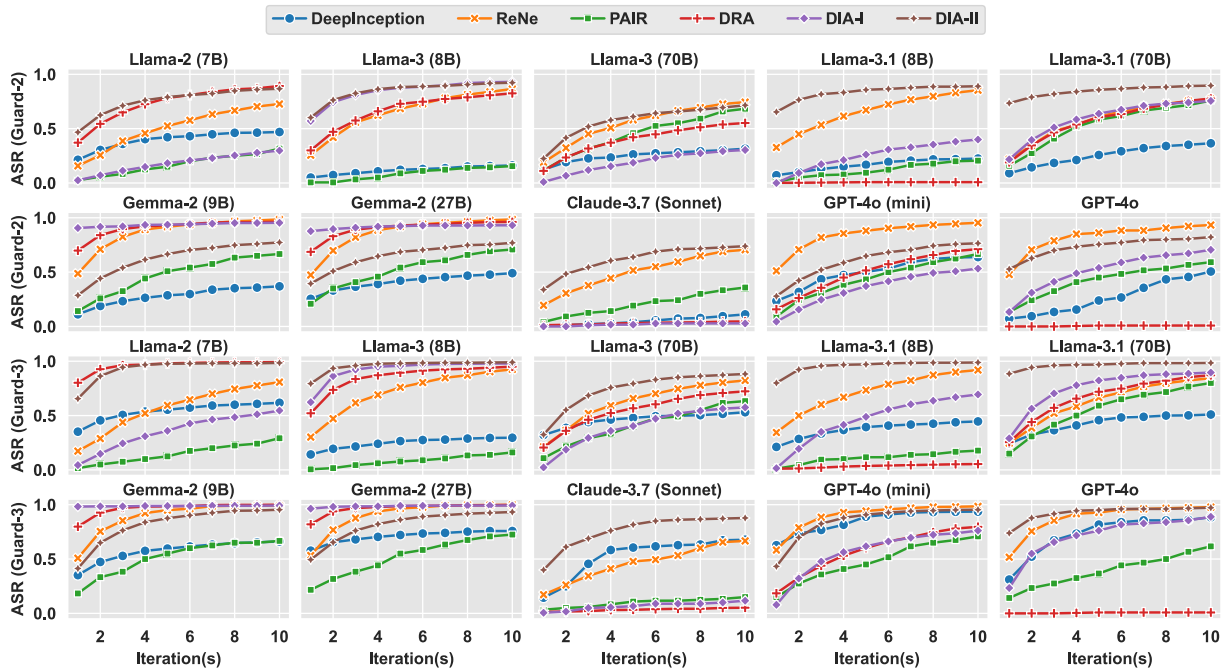
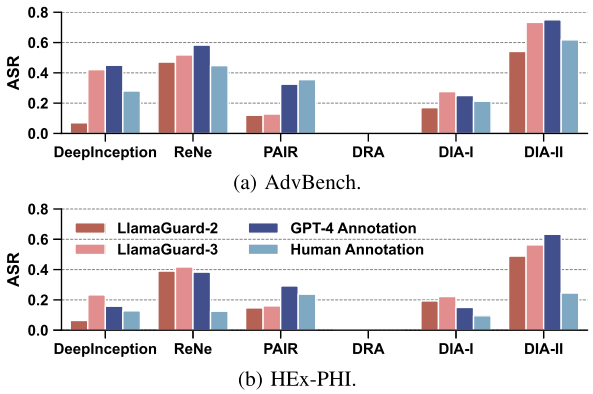


Fig. 6. Multi-query attack results on AdvBench.



(a) AdvBench.

(b) HEx-PHI.

Fig. 7. Comparison of different evaluators. Evaluated on GPT-4o's outputs.

TABLE V

MANUALLY TESTED ASR (AND ITS CORRESPONDING API TESTED ASR). WE CHOOSE LLAMAGUARD-3 AS THE EVALUATOR

Model	ASR (AdvBench)		ASR (HEx-PHI)		r
	DIA-I	DIA-II	DIA-I	DIA-II	
Attack OpenWebUI					
Llama-3	0.68 (0.615)	0.84 (0.790)	0.54 (0.545)	0.64 (0.654)	
Llama-3.1	0.02 (0.013)	0.86 (0.903)	0.00 (0.045)	0.62 (0.626)	0.987
Gemma-2	1.00 (0.985)	0.50 (0.410)	0.84 (0.931)	0.36 (0.402)	
Attack LobeChat					
Llama-3	0.64 (0.615)	0.70 (0.790)	0.64 (0.545)	0.76 (0.654)	
Llama-3.1	0.00 (0.013)	0.82 (0.903)	0.02 (0.045)	0.66 (0.626)	0.974
Gemma-2	0.96 (0.985)	0.50 (0.410)	0.90 (0.931)	0.46 (0.402)	
Attack Text Generation WebUI					
Llama-3	0.54 (0.615)	0.78 (0.790)	0.62 (0.545)	0.70 (0.654)	
Llama-3.1	0.00 (0.013)	0.86 (0.903)	0.04 (0.045)	0.70 (0.626)	0.986
Gemma-2	0.98 (0.985)	0.26 (0.410)	0.94 (0.931)	0.46 (0.402)	

of rewritten prompts with semantic similarities above a certain threshold. We can see that the prompts rewritten by ReNe exhibit low semantic similarities, with average values of 0.458 on AdvBench and 0.506 on HEx-PHI. In contrast, prompts rewritten using our prompt rewrite algorithm maintain higher average semantic similarities of 0.820 on AdvBench and 0.809 on HEx-PHI. As the semantic similarity threshold increases, ReNe's ASRs decrease, indicating that ReNe's improvement in ASRs comes at the cost of semantic integrity.

1) GPT-4 and Human Annotations: We evaluate a small subset of samples with GPT-4 and human annotators. Specifically, we randomly sample 100 attack instances from each dataset for every jailbreak attacker, resulting in $6 \times 2 \times 100 = 1,200$ samples. GPT-4 annotations follow the prompt setup in [63] with a temperature of 0. For human evaluation, five crowdworkers independently judge whether each attack succeeds based on the original instruction and model output, and we report the average ASR across them. The study was thoroughly reviewed and approved by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University, which serves as our IRB. Each human annotator is compensated USD 56.16 (≈USD 5.6 per hour, 30s per sample), exceeding the minimum wage in our hiring region. We compare GPT-4 and human annotations with LlamaGuard-2 and LlamaGuard-3 in Figure 7. Among all evaluators, DIA-II achieves the highest ASR. GPT-4 and LlamaGuard-3 show high consistency. Human annotations yield lower ASR on HEx-PHI, likely because its harmful instructions are embedded in complex scenes, requiring careful reading to detect.

2) Manual WebUI Attacks: To validate the practicality of dialogue injection, we manually test the three WebUIs with the highest star counts in Table II. We generate dialogue injection prompts and manually paste them into the WebUI input box. We evaluate three LLMs, Llama-3-8B, Llama-3.1-8B, and Gemma-2-9B, on each WebUI. Each ASR is computed from 50 pastes. We compare manually tested and API tested results in Table V. We further calculate the Pearson correlation

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

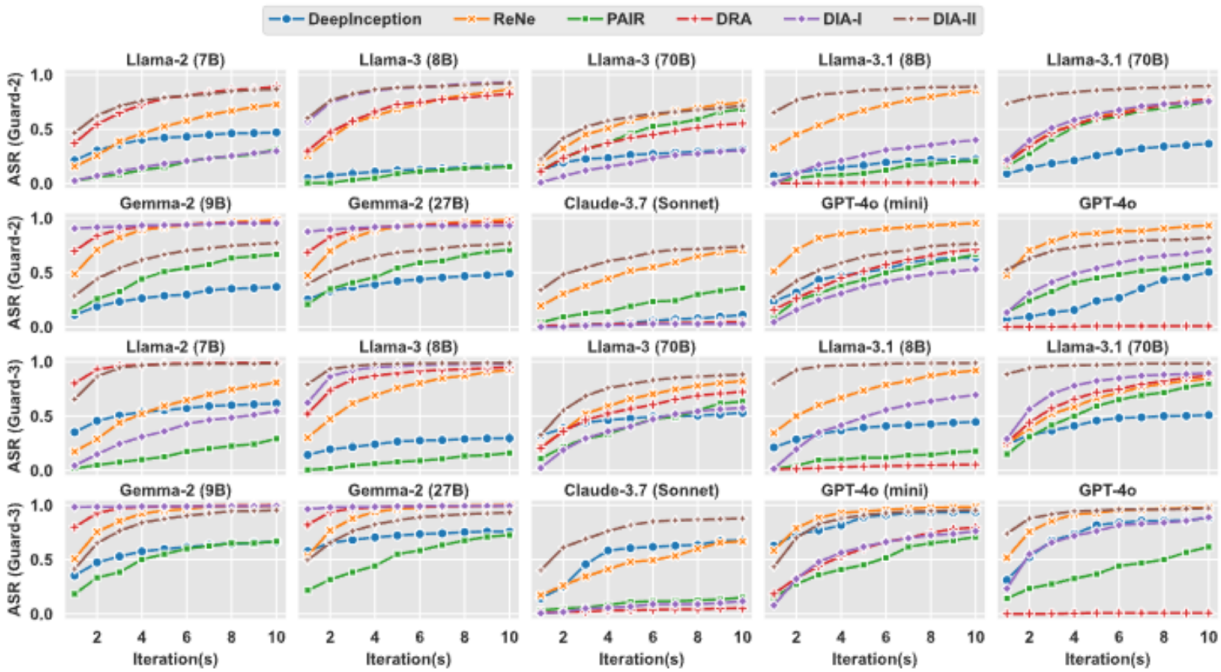


图 6. 在 AdvBench 上的多查询攻击结果。

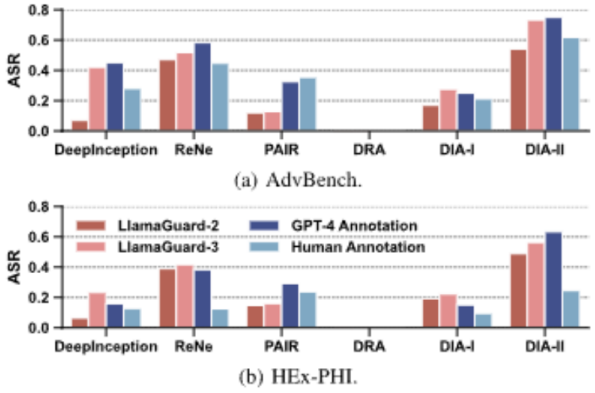


图 7. 不同评估器的比较。在 GPT-4o 的输出上进行评估。

表 V

MTASR (ICAPI TASR).
WCLG-3 E

Model	ASR (AdvBench)		ASR (HEx-PHI)		r
	DIA-I	DIA-II	DIA-I	DIA-II	
Attack OpenWebUI					
Llama-3	0.68 (0.615)	0.84 (0.790)	0.54 (0.545)	0.64 (0.654)	
Llama-3.1	0.02 (0.013)	0.86 (0.903)	0.00 (0.045)	0.62 (0.626)	0.987
Gemma-2	1.00 (0.985)	0.50 (0.410)	0.84 (0.931)	0.36 (0.402)	
Attack LobeChat					
Llama-3	0.64 (0.615)	0.70 (0.790)	0.64 (0.545)	0.76 (0.654)	
Llama-3.1	0.00 (0.013)	0.82 (0.903)	0.02 (0.045)	0.66 (0.626)	0.974
Gemma-2	0.96 (0.985)	0.50 (0.410)	0.90 (0.931)	0.46 (0.402)	
Attack Text Generation WebUI					
Llama-3	0.54 (0.615)	0.78 (0.790)	0.62 (0.545)	0.70 (0.654)	
Llama-3.1	0.00 (0.013)	0.86 (0.903)	0.04 (0.045)	0.70 (0.626)	0.986
Gemma-2	0.98 (0.985)	0.26 (0.410)	0.94 (0.931)	0.46 (0.402)	

具有高于特定阈值的语义相似性的重写提示。我们可以看到，ReNe 重写的提示表现出较低的语义相似性，在 AdvBench 上的平均值为 0.458，在 HEx-PHI 上的平均值为 0.506。相比之下，使用我们的提示重写算法重写的提示在 AdvBench 上保持了更高的平均语义相似性，分别为 0.820 和 0.809。随着语义相似性阈值的增加，ReNe 的 ASRs 下降，表明 ReNe 在 ASRs 上的改进是以牺牲语义完整性为代价的。

1) GPT-4 和人工标注：我们评估了一个小 GPT-4 和人类标注者的样本子集。具体来说，我们为每个越狱攻击者从每个数据集中随机抽取 100 个攻击实例，最终得到 6 × 2 × 100 = 1, 200 个样本。GPT-4 的标注遵循[63]中的提示设置，温度设置为 0。对于人工评估，五位众包工作者独立判断每个攻击是否成功，基于原始指令和模型输出，我们报告他们的平均 ASR。该研究经过伦理委员会的全面审查和批准。浙江大学生物医学工程与仪器科学学院，作为我们的伦理审查委员会（IRB）。每位人类标注员获得 56.16 美元的报酬（≈每小时 5.6 美元，每样本 30 秒），超过我们招聘地区的最低工资标准。我们在图 7 中比较了 GPT-4 和人类标注与 LlamaGuard-2 和 LlamaGuard-3 的结果。在所有评估者中，DIA-II 实现了最高的 ASR。GPT-4 和 LlamaGuard-3 显示出高度的一致性。人类标注在 HEx-PHI 上产生的 ASR 较低，可能是因为其有害指令嵌入在复杂场景中，需要仔细阅读才能检测到。

2) 手动 WebUI 攻击：为了验证对话注入的实用性，我们手动测试了表 II 中星级最高的三个 WebUI。我们生成对话注入提示，并手动将其粘贴到 WebUI 的输入框中。我们对每个 WebUI 评估了三个 LLMs：Llama-3-8B、Llama-3.1-8B 和 Gemma-2-9B。每个 ASR 值是根据 50 次粘贴计算得出的。我们在表 V 中比较了手动测试和 API 测试的结果。我们还计算了 Pearson 相关系数

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

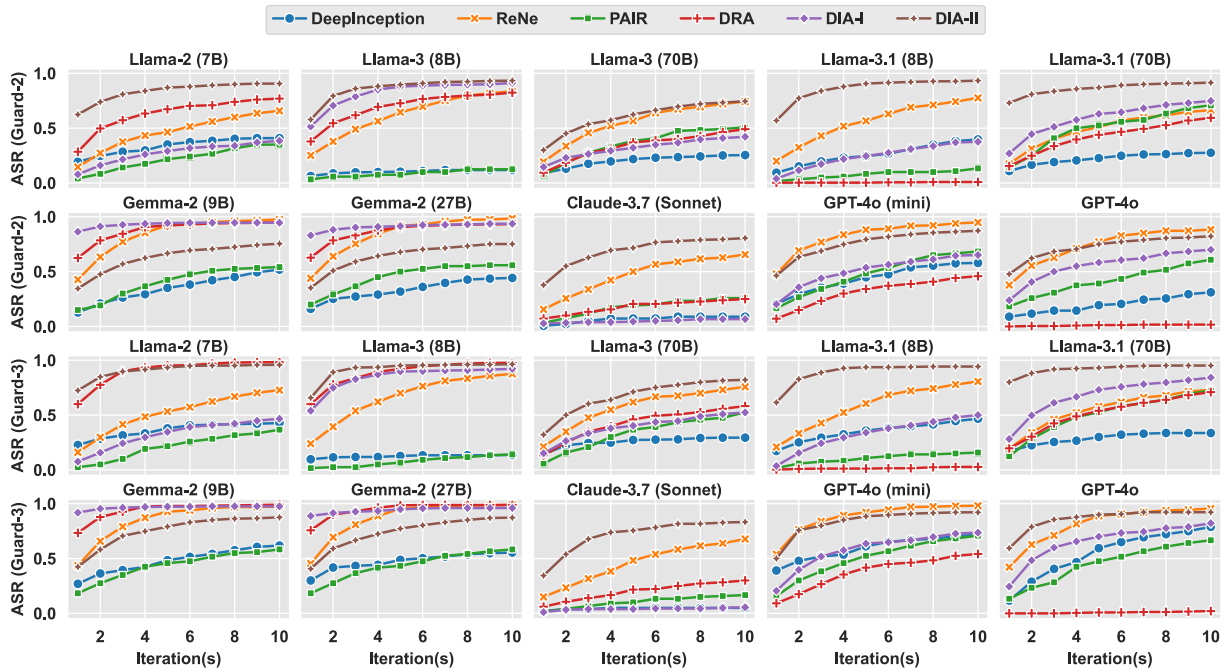


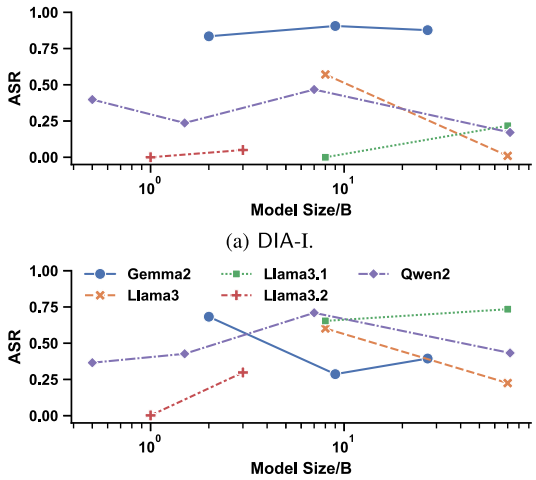
Fig. 8. Multi-query attack results on HEx-PHI.

coefficient r between the manual and API tested ASRs. All rs exceed 0.97. This strong correlation indicates that these WebUIs perform no input sanitization, and that our DIA achieves nearly identical effectiveness when executed through the WebUI and through the API.

C. Multi-Query Jailbreak Attacks

In this section, we assess changes in ASR versus query iterations. For our DIA, we rewrite the original malicious prompts after each query. During each rewrite, we randomly select a number from $[1, 2, 3]$ with probabilities $[0.3, 0.5, 0.2]$ to determine the number of basic operations, $N_{\mathcal{S}}$, then sample $N_{\mathcal{S}}$ operations from the basic operation set $\mathcal{O}$ to compose the rewrite strategy, $\mathcal{S}$. The evaluation results on AdvBench and HEx-PHI are presented in Figure 6 and Figure 8, respectively.

We observe a steady increase in ASRs of both DIA-I and DIA-II with the increase of query iterations. When targeting Llama-3.1-8B and GPT-4o-mini, DIA exhibits single-query ASRs of less than 0.1 judged by LlamaGuard-2. After 10 iterations, the ASRs for DIA-I on Llama-3.1-8B exceed 0.4. And on GPT-4o-mini, the ASRs exceed 0.5. In contrast, DRA fails on Llama-3.1-8B and GPT-4o-mini, with ASRs remaining near zero. Our experiments suggest that Llama-3.1-8B is the most secure model tested in this paper. Aside from ReNe and our DIA, other jailbreak attacks demonstrate significantly lower ASRs on Llama-3.1-8B. Notably, our measured ASRs for PAIR on Llama-2-7B are higher than those reported in the original paper. Our tested ASR on AdvBench is 0.317 after 10 iterations judged by LlamaGuard-2, significantly outperforming the original paper's ASR of 0.040. We believe this is because we use a more powerful attacker LLM, Gemma-2-27B, compared to Mixtral-8 $\times$ 7B used in the original paper. The former model's AlpacaEval 2.0 score is 51.9, and the latter's is 23.7. Despite this, DIA still outperforms PAIR in most cases.



(a) DIA-I.

(b) DIA-II.

Fig. 9. ASRs on AdvBench versus model size.

D. Further Analysis

1) Analysis of Model Size: Previous work often overlooked the impact of model size on jailbreak attacks due to the complexity of controlling other variables. Different-sized LLMs may employ distinct alignment strategies. In this section, we address this issue by assuming that models within the same family utilize consistent alignment strategies, as they are released by the same developer team during the same period. Figure 9 illustrates the ASRs on AdvBench of DIA versus model size. Intuitively, one might assume that a larger LLM, with superior language comprehension abilities, would more easily detect and resist jailbreak prompts. However, as depicted

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

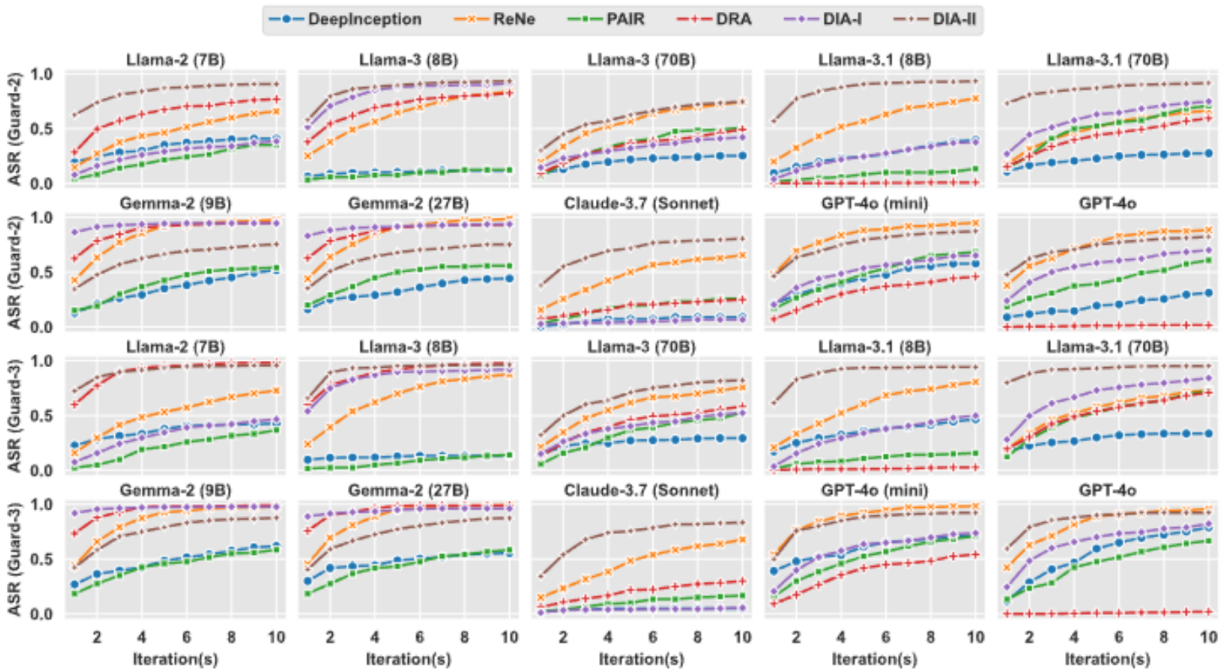


图 8. HEx-PHI 上的多查询攻击结果。

手动测试和 API 测试的 ASR 之间的系数 r。所有 r 值均超过 0.97。这种强相关性表明这些 WebUI 没有进行输入清理，并且我们的 DIA 在通过 WebUI 和 API 执行时几乎达到了相同的有效性。

C. 多查询越狱攻击

在本节中，我们评估 ASR 随查询迭代的变化。对于我们的 DIA，我们在每次查询后重写原始恶意提示。在每次重写过程中，我们根据概率 [0.3, 0.5, 0.2] 随机选择一个数字 [1, 2, 3] 来确定基本操作的数量 N，然后从基本操作集 O 中采样 N 个操作来组成重写策略 S。AdvBench 和 HEx-PHI 上的评估结果分别如图 6 和图 8 所示。

我们观察到 DIA-I 和 DIA-II 的 ASR 随着查询迭代次数的增加而稳定上升。在针对 Llama-3.1-8B 和 GPT-4o-mini 时，DIA 在 LlamaGuard-2 的判断下表现出单次查询 ASR 低于 0.1。在 10 次迭代后，Llama-3.1-8B 上的 DIA-I ASR 超过 0.4，而在 GPT-4o-mini 上，ASR 超过 0.5。相比之下，DRA 在 Llama-3.1-8B 和 GPT-4o-mini 上失败，ASR 保持在接近零的水平。我们的实验表明，Llama-3.1-8B 是本文测试中最安全的模型。除了 ReNe 和我们的 DIA 之外，其他越狱攻击在 Llama-3.1-8B 上的 ASR 显著较低。值得注意的是，我们在 Llama-2-7B 上测量的 PAIR ASR 高于原始论文中报告的数值。在 LlamaGuard-2 的判断下，我们在 AdvBench 上测试的 ASR 在 10 次迭代后为 0.317，显著优于原始论文中 0.040 的 ASR。我们认为这是因为我们使用了比原始论文中使用的 Mixtral-8 × 7B 更强大的攻击者 LLM，Gemma-2-27B。前者的 AlpacaEval 2.0 得分为 51.9，

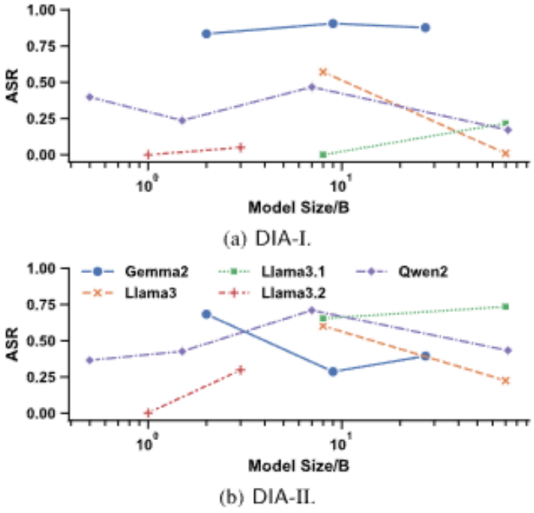


图 9. AdvBench 上的 ASR 与模型大小关系。

而后者为 23.7。尽管如此，DIA 在大多数情况下仍然优于 PAIR。

D. 进一步分析

1) 模型大小分析：以往研究往往由于控制其他变量的复杂性而忽略了模型大小对越狱攻击的影响。不同大小的 LLMs 可能会采用不同的对齐策略。在本节中，我们通过假设同一系列的模型在同一时期由同一开发团队发布，因此采用一致的对齐策略来解决这一问题。图 9 展示了 DIA 在 AdvBench 上的 ASR 与模型大小的关系。直观上，人们可能会假设一个具有更强语言理解能力的更大的 LLM 会更容易检测并抵抗越狱提示。然而，如图所示

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

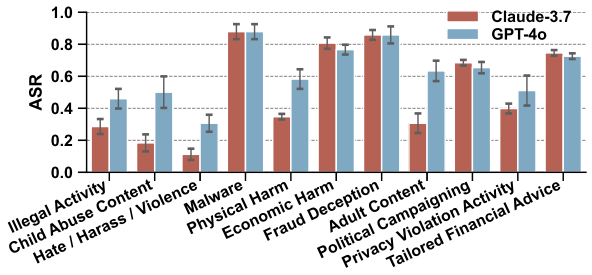


Fig. 10. ASR comparison across different prompt categories.

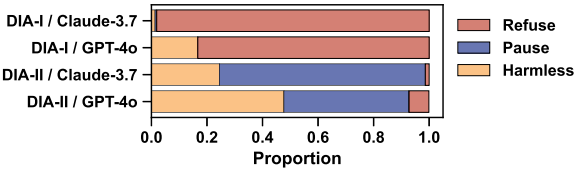


Fig. 11. Distribution of failure types for DIA-I and DIA-II across GPT-4o and Claude-3.7.

in Figure 9, aside from the Llama-3 family, which aligns with this hypothesis, other LLM families exhibit a trend where the ASR increases with model size. This phenomenon indicates that larger LLMs are more susceptible to jailbreak attacks. We attribute this phenomenon to the inherent trade-off between usefulness and safety in LLMs [64], [65], [66]. The essence of black-box jailbreak attacks is exploiting the usefulness of LLMs to compromise their safety. It is evident that increasing the size of an LM enhances its usefulness [1], [62]. Therefore, when employing the same alignment strategy, the increased usefulness of a larger LM may encroach upon its safety features. As for the contrary behavior observed in the Llama-3 family, we believe it results from the knowledge cutoff discrepancy [67]; Llama-3-70B has a cutoff in December 2023, while Llama-3-8B's is in March 2023. Llama-3-70B may utilize an updated alignment dataset.

2) Analysis of Prompt Categories: We further analyze how different prompt categories affect LLMs' vulnerability to jailbreak attacks. For this purpose, we adopt the 11 predefined prompt categories provided by HEx-PHI, each containing 30 instructions. We evaluate the ASR of Claude-3.7-Sonnet and GPT-4o across these categories using LlamaGuard-3 as the evaluator to ensure consistent safety assessment. As shown in the Figure 10, ASR varies significantly across categories, indicating that jailbreak resistance strongly depends on prompt semantics. Both models exhibit high ASRs in Malware, Economic Harm, Fraud Deception, and Tailored Financial Advice, suggesting that instructions involving contextual reasoning or persuasive intent more easily bypass alignment filters. Conversely, categories such as Illegal Activity, Child Abuse Content, and Hate/Harass/Violence yield lower ASRs, reflecting more robust refusal mechanisms against explicitly illegal or socially prohibited topics. These findings highlight that jailbreak susceptibility is category-dependent, and that improving LLM safety requires fine-grained, domain-specific alignment rather than uniform safety tuning.

3) Analysis of Failure Types: To better understand model behaviors under different attack settings, we analyze the failure types on the HEx-PHI. After identifying incorrect outputs using LlamaGuard-3, we classify them into three categories as shown in Figure 11: Refuse (the model explicitly rejects to answer), Pause (the model output is interrupted or incomplete), and Harmless (the model produces a complete response that LlamaGuard-3 judges to be safe). As illustrated in Figure 11, the distribution of failure types differs substantially between DIA-I and DIA-II. For both Claude-3.7-Sonnet and GPT-4o, the proportion of Refuse failures significantly decreases in DIA-II compared with DIA-I, while the share of Pause and Harmless cases increases. This trend indicates that the deferred answering strategy effectively mitigates the models' tendency to reject responses outright, thereby enhancing their responsiveness to adversarial prompts.

TABLE VI

ABLATION STUDY OF DIA'S DIALOGUE COMPONENTS. WE REMOVE DIALOGUE COMPONENTS AND MEASURE THE ASRS ON ADVBENCH JUDGED BY LLAMAGUARD-2

	DIA-I			DIA-II		
Model	Llama-2	Llama-3	Gemma-2	Llama-2	Llama-3	Gemma-2
original attack	**0.025**	**0.571**	**0.906**	**0.467**	**0.602**	0.287
w/o system	0.000	0.010	0.869	0.448	0.502	0.213
w/o hypnosis	0.002	0.365	0.898	0.465	0.590	**0.291**
w/o guidance	0.000	0.144	0.798	0.113	0.554	0.102

E. Ablation Study

1) Ablation of Dialogue Components: In addition to the payload, DIA dialogues contain multiple components, namely system prompt replacement, hypnosis, and answer guidance. To evaluate the efficacy of these components, we remove each one and compare the resulting ASR with that of the full dialogues. We conduct this ablation study using Llama-2-7B, Llama-3-8B, and Gemma-2-9B, with the evaluated ASRs presented in Table VI. Notably, Gemma-2's chat template does not support the system role. During attacks, we incorporate the replaced system prompt with the user text.

From Table VI, we can see that removing dialogue components generally results in a decrease in ASR, with the only exception of DIA-II targeting Gemma-2-9B, where removing hypnosis yields a slight increase in ASR by 0.004. We hypothesize this is due to the inclusion of malicious content in the hypnosis component, which compromises DIA-II's disguise. We also observe differing sensitivities to the system prompt between DIA-I and DIA-II. For DIA-I, removing the system prompt when attacking Llama-2-7B and Llama-3-8B leads to a more than 95% drop in ASR. This is likely because DIA-I incorporates malicious prompts directly into the input, relying on the system prompt to lower the victim LLM's guard. Both DIA-I and DIA-II experience significant ASR reductions upon removing answer guidance, indicating that our answer guidance effectively reduces the probability of the victim LLM producing a bad answer.

2) Ablation of Prompt Rewriting: Section V-C demonstrates that DIA's ASR increases with the number of query iterations. There are two contributions for the increase: the inherent randomness of LLM outputs, and the effectiveness of our prompt rewrite algorithm. To validate the effectiveness

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

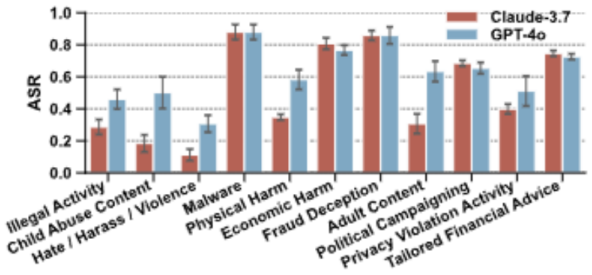


图 10. 不同提示类别中的语音识别（ASR）比较。

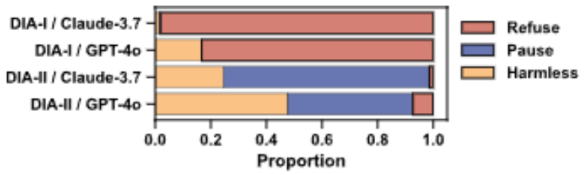


图 11. DIA-I 和 DIA-II 在 GPT-4o 和 Claude-3.7 中的失败类型分布。

在图 9 中，除了 Llama-3 系列（这与该假设一致）之外，其他 LLM 系列表现出随着模型尺寸增大，语音识别（ASR）增加的趋势。这种现象表明，更大的 LLM 更容易受到越狱攻击。我们将这种现象归因于 LLM 中有用性与安全性之间的固有权衡 [64], [65], [66]。黑盒越狱攻击的本质是利用 LLM 的有用性来损害其安全性。很明显，增加 LM 的大小会提高其有用性 [1], [62]。因此，在采用相同的对齐策略时，更大 LM 的有用性增加可能会损害其安全特性。至于 Llama3 系列的相反行为，我们认为这是由于知识截止日期差异 [67] 造成的；Llama-3-70B 的截止日期是 2023 年 12 月，而 Llama-3-8B 的截止日期是 2023 年 3 月。Llama-3-70B 可能使用了更新的对齐数据集。

2) 提示类别的分析：我们进一步分析
如何不同的提示类别影响 LLMs 对越狱攻击的易受性。为此，我们采用了 HEx-PHI 提供的 11 个预定义提示类别，每个类别包含 30 条指令。我们使用 LlamaGuard-3 作为评估器，在这些类别中评估 Claude-3.7-Sonnet 和 GPT-4o 的 ASR，以确保一致的安全评估。如图 10 所示，ASR 在不同类别中差异显著，表明越狱抵抗能力强烈依赖于提示的语义。两种模型在恶意软件、经济损害、欺诈欺骗和定制财务建议类别中表现出高 ASR，这表明涉及情境推理或说服意图的指令更容易绕过对齐过滤器。相反，非法活动、儿童虐待内容和仇恨/骚扰/暴力等类别产生较低的 ASR，反映出对明确非法或社会禁止话题的拒绝机制更为牢固。这些发现表明，越狱易受性是类别相关的，改进 LLM 的安全性需要细粒度、特定领域的对齐，而不是统一的安全调整。

3) 失败类型分析：为了更好地理解模型
在不同攻击设置下的行为，我们分析了失败情况

表 VI

ASDIA’ DC. WR
DCMASRAB

JLG-2

	DIA-I			DIA-II		
Model	Llama-2	Llama-3	Gemma-2	Llama-2	Llama-3	Gemma-2
original attack	**0.025**	**0.571**	**0.906**	**0.467**	**0.602**	0.287
w/o system	0.000	0.010	0.869	0.448	0.502	0.213
w/o hypnosis	0.002	0.365	0.898	0.465	0.590	**0.291**
w/o guidance	0.000	0.144	0.798	0.113	0.554	0.102

在 HEx-PHI 上输入类型。在通过 LlamaGuard-3 识别出错误输出后，我们将它们分为三类，如图 11 所示：拒绝（模型明确拒绝回答）、暂停（模型输出中断或不完整）和无害（模型生成完整响应，LlamaGuard-3 判断为安全）。如图 11 所示，DIA-I 和 DIA-II 的失败类型分布差异很大。对于 Claude-3.7-Sonnet 和 GPT-4o，与 DIA-I 相比，DIA-II 中拒绝失败的占比显著减少，而暂停和无害案例的占比增加。这一趋势表明，延迟回答策略有效缓解了模型直接拒绝响应的倾向，从而增强了它们对对抗性提示的响应能力。

E. 消融研究

1) 对话组件的消融：除了
负载，DIA 对话包含多个组件，即系统提示替换、催眠和答案指导。为了评估这些组件的有效性，我们移除每一个组件，并将结果 ASR 与完整对话的 ASR 进行比较。我们使用 Llama-2-7B、Llama-3-8B 和 Gemma-2-9B 进行这项消融研究，评估的 ASR 结果展示在表 VI 中。值得注意的是，Gemma-2 的聊天模板不支持系统角色。在攻击过程中，我们将替换后的系统提示与用户文本结合。

从表 VI 可以看出，移除对话组件通常会降低 ASR，唯一的例外是针对 Gemma-2-9B 的 DIA-II，移除催眠内容会导致 ASR 略微提升 0.004。我们推测这是因为催眠组件中包含了恶意内容，从而破坏了 DIA-II 的伪装。我们还观察到 DIA-I 和 DIA-II 对系统提示的敏感度不同。对于 DIA-I，在攻击 Llama-2-7B 和 Llama-3-8B 时移除系统提示会导致 ASR 下降超过 95%。这可能是由于 DIA-I 将恶意提示直接嵌入输入中，依赖系统提示来降低目标 LLM 的警惕性。移除答案指导后，DIA-I 和 DIA-II 的 ASR 均显著降低，表明我们的答案指导有效降低了目标 LLM 生成错误答案的概率。

2) 提示重写的移除：第 V-C 节说明
表明 DIA 的语音识别率随着查询迭代次数的增加而提高。这种增加有两个贡献：一是 LLM 输出的固有随机性，二是我们提示重写算法的有效性。为了验证这种有效性

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

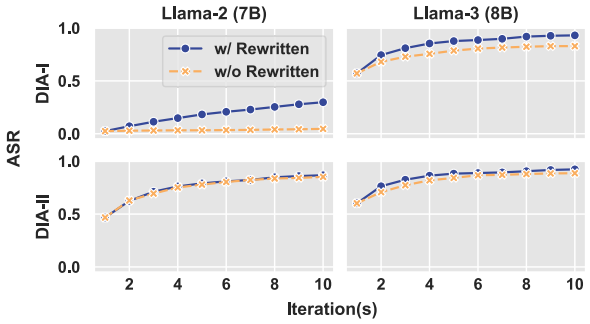
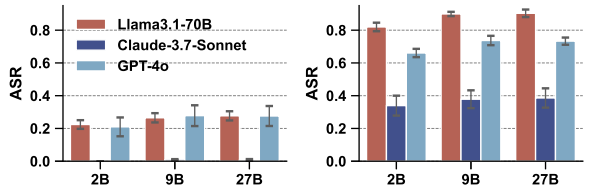


Fig. 12. Ablation study of DIA's prompt rewrite algorithm. The results are tested on AdvBench judged by LlamaGuard-2.



(a) DIA-I. (b) DIA-II.

Fig. 13. Ablation study of the size of auxiliary model. The results are tested on AdvBench judged by LlamaGuard-3.

of our prompt rewrite algorithm, we test multi-query attacks without the prompt rewrite mechanism on AdvBench. the results are shown in Figure 12.

We observe that both DIA-I and DIA-II exhibit slower ASR growth in the absence of the rewrite mechanism. This effect is particularly noticeable with DIA-I when targeting Llama-2-7B, where removing the rewrite mechanism results in ASRs remaining near zero. In experiments with Llama-2-7B and Llama-3-8B, the prompt rewrite mechanism contributes more significantly to improvements in DIA-I than in DIA-II. We attribute this to DIA-I's strategy of directly using malicious prompts in the input. Our prompt rewrite operation can reduce the apparent maliciousness of these prompts by adding reasons and reconstructing syntax.

3) Ablation of Auxiliary Model: Our ABGM and SDGM rely on an auxiliary model π_{aux} to generate affirmative beginnings and keywords-to-alternatives mappings. In the main experiments, we set π_{aux} to Gemma-2-27B. To evaluate performance under resource-constrained scenarios, we replace it with smaller LLMs. The ASR comparison is presented in Figure 13.

Larger models yield higher ASR. Specifically, using Gemma-2-2B results in an average ASR drop of 14.7% compared with Gemma-2-27B, while Gemma-2-9B exhibits a 2.4% decrease. The reduction mainly stems from weaker instruction-following abilities in smaller models, causing output-format mismatches and extraction errors. Nevertheless, the 9B model achieves performance comparable to the 27B variant. As smaller models continue to improve in instruction adherence, we believe DIA can be effectively applied in more low-compute environments.

F. Attack Against Defenses

Existing jailbreak defenses can be categorized by their stage of operation into pre-generation, generation-time, and post-generation defenses. Pre-generation defenses inspect user prompts and block those identified as harmful; their effectiveness is independent of the underlying generative model. Generation-time defenses modify the original prompt or append extra prompts to prevent the model from producing unsafe outputs. Post-generation defenses, in contrast, analyze the generated text after it is completed. We exclude post-generation defenses in our defense testing, as they are not real-time. Modern LLM chat services need to support *streaming generation* to enhance user experience. In streaming mode, the inference engine decodes tokens while simultaneously transmitting previously generated tokens to the chat interface. This feature is particularly valuable for long responses, allowing users to read or terminate early, which saves computation and time. As mainstream systems like ChatGPT and Claude utilize streaming generation, post-generation defenses are incompatible with this paradigm.

In particular, we use the following six defenses to evaluate DIA and baselines' defense penetrating capabilities.

- *OpenAI Moderation* [68] is a free API to check whether text or images are potentially harmful. We employ it as a pre-generation filter, where only the input prompt is subjected to safety detection. The exact version used is omini-moderation-2024-09-26.
- *Perplexity Filter* [69] filters input prompts that have higher perplexity values than a threshold. It is effective in defending against white-box jailbreak attacks, which often add meaningless text. We set the threshold to the maximum perplexity found in the AdvBench and HEx-PHI datasets, and use Gemma-2-2B to calculate perplexity.
- *Defensive System Prompt* [70] optimizes a safe system prompt through prompt-tuning, which directs LLMs to prioritize safety and prevent the generation of harmful outputs. We employ the official Llama-2 safety prompt as the initial system prompt.
- *Defensive Prompt Patch* [57] appends a suffix prompt to every input, which reminds LLMs to rethink the user's input and refuse harmful responses. We test the prompt patches proposed in [57] for Llama-2 and Mistral and report the best defense results.
- *Bergeron* [71] employs the secondary LLM to monitor and correct LLM's input and output. We use the victim LLM itself as the secondary LLM. Bergeron spans all three defense stages: it first detects whether the input prompt exhibits potential policy violations; if so, it invokes the secondary LLM to revise the input; finally, it inspects the generated output for policy compliance. We only use Bergeron's first two stages because the output inspection process is shown to be slow and impractical [15].
- *Input Canonicalization* is an adaptive defense to DIA. It filters out special-token texts in the input, breaking the chat template structure of DIA prompts. For Claude and GPT, whose chat templates are proprietary, we construct DIA prompts using only role words such as "user" and "assistant" without special tokens.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

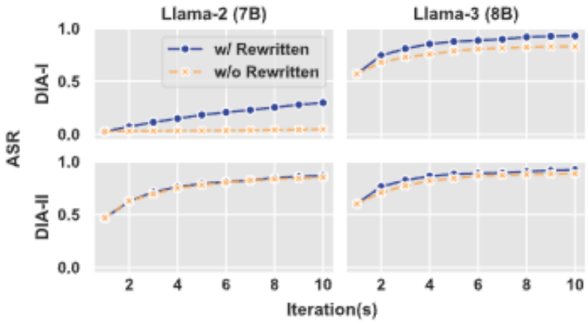


图 12. DIA 提示重写算法的消融研究。结果在由 LlamaGuard-2 判定的 AdvBench 上测试。

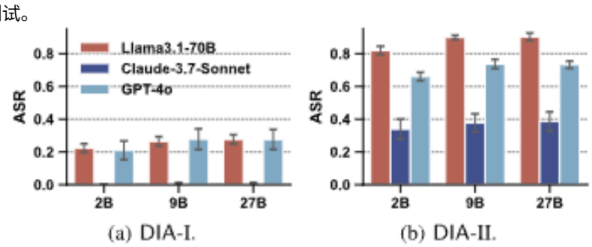


图 13. 辅助模型大小的消融研究。结果在由 LlamaGuard-3 判定的 AdvBench 上测试。

对于我们的提示重写算法，我们在 AdvBench 上测试了没有提示重写机制的多查询攻击。结果如图 12 所示。

我们观察到，在缺少重写机制的情况下，DIA-I 和 DIA-II 的 ASR 增长都变慢。这种效应在针对 Llama-2-7B 的 DIA-I 中尤为明显，移除重写机制后，ASR 保持在接近零的水平。在 Llama-2-7B 和 Llama-3-8B 的实验中，提示重写机制对 DIA-I 的提升比 DIA-II 更为显著。我们归因于 DIA-I 的策略是在输入中直接使用恶意提示。我们的提示重写操作可以通过添加理由和重建句法来降低这些提示的明显恶意性。

3) 辅助模型的移除：我们的 ABGM 和 SDGM 依赖于一个辅助模型π来生成肯定的开头和关键词到替代词的映射。在主要实验中，我们将π设置为 Gemma-2-27B。为了评估资源受限场景下的性能，我们用较小的 LLMs 来替换它。ASR 的比较展示在图 13 中。

较大的模型产生更高的 ASR。具体来说，使用 Gemma-2-2B 导致与 Gemma-2-27B 相比平均 ASR 下降了 14.7%，而 Gemma-2-9B 表现出 2.4%的下降。这种减少主要源于较小模型在指令遵循能力上的不足，导致输出格式不匹配和提取错误。尽管如此，9B 模型实现了与 27B 变体相当的性能。随着较小模型在指令遵循上的持续改进，我们相信 DIA 可以在更多低计算环境中有效应用。

F. 对防御的攻击

现有的越狱防御措施可以根据其操作阶段分为预生成、生成时和生成后防御。生成前防御检查用户提示并阻止被识别为有害的提示；其有效性独立于底层生成模型。生成时防御修改原始提示或附加额外提示以防止模型生成不安全输出。相比之下，生成后防御在文本生成完成后分析生成的文本。我们在防御测试中排除生成后防御，因为它们不是实时防御。现代 LLM 聊天服务需要支持流式生成以增强用户体验。在流式模式下，推理引擎在解码标记的同时将先前生成的标记传输到聊天界面。此功能对于长响应特别有价值，允许用户阅读或提前终止，从而节省计算和时间。由于像 ChatGPT 和 Claude 这样的主流系统使用流式生成，生成后防御与此范式不兼容。

我们特别使用以下六种防御措施来评估 DIA 和基线防御穿透能力。

- OpenAI Moderation [68] 是一个免费的 API，用于检查文本或图像是否具有潜在危害性。我们将其用作预生成过滤器，仅对输入提示进行安全检测。使用的确切版本是 omni-moderation-2024-09-26。

- Perplexity Filter [69] 过滤具有高于阈值的困惑度值的输入提示。它在防御白盒越狱攻击方面非常有效，这类攻击通常添加无意义的文本。我们将阈值设置为 AdvBench 和 HEx-PHI 数据集中发现的最高困惑度值，并使用 Gemma-2-2B 计算困惑度。

- Defensive System Prompt [70] 通过提示调整优化安全系统提示，指导 LLMs 优先考虑安全性并防止生成有害输出。我们使用官方 Llama-2 安全提示作为初始系统提示。

- 防御性提示补丁 [57] 在每个输入后附加一个后缀提示，提醒 LLMs 重新思考用户的输入并拒绝有害的响应。我们测试了[57]中提出的提示补丁，用于 Llama-2 和 Mistral，并报告了最佳的防御结果。

- Bergeron [71] 采用一个次级 LLM 来监控和纠正 LLM 的输入和输出。我们使用受害者 LLM 本身作为次级 LLM。Bergeron 涵盖了所有三个防御阶段：它首先检测输入提示是否表现出潜在的策略违规；如果是，它调用次级 LLM 来修改输入；最后，它检查生成的输出是否符合策略。我们只使用 Bergeron 的前两个阶段，因为输出检查过程被证明是缓慢且不切实际的[15]。

- 输入规范化是对 DIA 的一种自适应防御。它过滤掉输入中的特殊标记文本，破坏了 DIA 提示的聊天模板结构。对于 Claude 和 GPT，其聊天模板是专有的，我们仅使用“用户”和“助手”等角色词来构建 DIA 提示，而不使用特殊标记。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

TABLE VII

DEFENSE PASS RATE OF DIA AND BASELINES. THE RESULTS ARE TESTED ON ADVBENCH JUDGED BY LLAMAGUARD-2. BOLD DENOTES THE BEST RESULT, UNDERLINE SIGNIFIES THE RUNNER-UP. WE EXCLUDE CASES WITH AN ASR LOWER THAN 0.04, BECAUSE THE LIMITED NUMBER OF SUCCESSFUL ATTACKS LEADS TO UNRELIABLE DPR COMPUTATION

Attacker	OpenAI	Perplexity	System			Patch			Bergeron			Canonicalize		
			Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT
DeepInception	0.054	**1.000**	0.634	–	0.000	0.893	–	0.154	0.018	–	0.000	–	–	–
ReNe	0.655	**1.000**	0.277	0.200	**0.116**	0.869	0.573	**0.870**	0.300	0.000	0.023	–	–	–
PAIR	0.585	**1.000**	0.775	0.000	0.063	0.937	0.595	0.501	0.120	0.000	0.063	–	–	–
DRA	0.587	**1.000**	0.609	–	–	0.898	–	–	0.460	–	–	–	–	–
DIA-I	**1.000**	**1.000**	**1.000**	–	0.000	**0.985**	–	0.727	0.651	–	0.150	0.989	–	0.024
DIA-II	0.725	**1.000**	0.798	**0.298**	0.032	0.871	**1.000**	0.846	**0.690**	**0.039**	**0.227**	**1.000**	1.000	**0.623**

We employ Defense Pass Rate (DPR) from [72] as the metric, which is the percentage of successful jailbreaking prompts that can bypass the defenses. We conduct tests on AdvBench using three LLMs: Gemma-2-9B, Claude-3.7-Sonnet, and GPT-4o. We choose Gemma-2-9B because all jailbreak attackers, including DIA and baselines, achieve notable ASRs on Gemma-2-9B, which helps us compute precise DPRs. The results are shown in Table VII.

Our DIA achieves superior DPRs over baselines in most cases. The final part of DIA's prompt, which is an answer guidance prompt, asks the assistant to refine its previous response, effectively masking the jailbreak intent. This enables DIA to circumvent OpenAI moderation and Bergeron detection. We also discover that the perplexity filter is completely ineffective against black-box jailbreak attacks, indicating that all black-box jailbreak prompts are highly readable. System prompts are more effective than prompt patches, particularly for proprietary models. This may be attributed to the higher weighting of system prompts during the alignment process of such models. In addition, input canonicalization proves ineffective in defending against DIA, suggesting that jailbreaks can still be enabled even when only role tags are present.

VI. RELATED WORK

A. LLM Jailbreak Attacks

Existing research on LLM jailbreak attacks can be categorized into white-box, gray-box, and black-box attacks, based on the level of access available to the attacker. White-box attacks involve model training. GCG [27] optimizes an adversarial suffix using a greedy gradient-based search. AutoDAN [73] enhances prompt readability with dual objectives. Gray-box attacks require local inference. A common type of gray-box attack is the prefilling attack [28], [29], which deposits an affirmative beginning in the response. Black-box attacks only need query access to the victim model. DeepInception [14] employs imaginary scenes to hypnotize, and DRA [15] conceals sensitive words. LLM-Fuzzer [74] mimics fuzz testing for automated jailbreak template generation. All the above attacks focus on single-turn dialogue. Crescendo [19] proposed a multi-turn jailbreak but is limited to designing only user prompts. Our DIA makes the first step towards exploiting the entire dialogue history, including user prompts, system prompts, and assistant responses.

B. LLM Jailbreak Defenses

Research on LLM jailbreak defenses is generally divided into learning-based and strategy-based approaches. Learning-based defenses enhance model robustness through tailored training methods. For instance, DRO [70] and DDP [57] employ robust optimization to generate reminding prompts, while MART [75] introduces a multi-round adversarial training framework that strengthens model resistance. Strategy-based defenses rely on operational techniques that help models identify and counteract malicious inputs or outputs. Perplexity filter [69] rejects inputs with high perplexities. Bergeron [71] and SelfDefense [76] leverage the second LLM to monitor inputs or outputs. RAIN [77] employs forward-looking searches into generation, allowing the LLM to evaluate its output by itself. Despite these advances, our proposed attack method, which exploits historical dialogue manipulation, demonstrates the need for more comprehensive defense strategies that account for multi-turn interactions.

VII. CONCLUSION

In this paper, we introduce DIA, a novel black-box jailbreak paradigm that leverages the structure of LLM chat templates and carefully crafted dialogues to exploit vulnerabilities in large language models. By manipulating dialogue history, DIA exposes new vulnerabilities in LLM safety mechanisms. Our findings highlight the critical need to consider historical dialogue interactions when developing security measures for LLMs, emphasizing the ongoing challenges in securing LLMs against adversarial exploitation.

ETHICS CONSIDERATION

This research aims to highlight security vulnerabilities in LLMs to strengthen defenses against adversarial misuse. All experiments were conducted in controlled environments, and no harmful content was shared with individuals unrelated to the research. The human evaluation study was reviewed and approved by the Ethics Committee of the College of Biomedical Engineering and Instrument Science, Zhejiang University. The authors believe responsible disclosure of these vulnerabilities is crucial for advancing AI security and developing robust safeguards.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers and the editors for their valuable comments and constructive suggestions.

REFERENCES

[1] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[2] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2022.
[3] Q. Gu, "LLM-based code generation method for golang compiler testing," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 2201–2203.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

表 VII

DPRDIA B. TRATABJLG-2. BDB R, USR-. WECWASR LT0.04, BLN SALUDPR C

Attacker	OpenAI	Perplexity	System			Patch			Bergeron			Canonicalize		
			Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT	Gemma	Claude	GPT
DeepInception	0.054	**1.000**	0.634	–	0.000	0.893	–	0.154	0.018	–	0.000	–	–	–
ReNe	0.655	**1.000**	0.277	0.200	**0.116**	0.869	0.573	**0.870**	0.300	0.000	0.023	–	–	–
PAIR	0.585	**1.000**	0.775	0.000	0.063	0.937	0.595	0.501	0.120	0.000	0.063	–	–	–
DRA	0.587	**1.000**	0.609	–	–	0.898	–	–	0.460	–	–	–	–	–
DIA-I	**1.000**	**1.000**	**1.000**	–	0.000	**0.985**	–	0.727	0.651	–	0.150	0.989	–	0.024
DIA-II	0.725	**1.000**	0.798	**0.298**	0.032	0.871	**1.000**	0.846	**0.690**	**0.039**	**0.227**	**1.000**	1.000	**0.623**

我们采用来自 [72] 的防御通过率 (DPR) 作为指标，该指标是能够绕过防御的成功越狱提示的百分比。我们在 AdvBench 上使用三个 LLM 进行测试：Gemma-2-9B、Claude-3.7-Sonnet 和 GPT-4o。我们选择 Gemma-2-9B，因为所有越狱攻击者（包括 DIA 和基线）在 Gemma-2-9B 上都取得了显著的 ASR，这有助于我们计算精确的 DPR。结果如表 VII 所示。

我们的 DIA 在大多数情况下相较于基线实现了更优的 DPR。DIA 提示语的最后一部分，即答案指导提示语，要求助手完善其之前的响应，有效地掩盖了越狱意图。这使得 DIA 能够绕过 OpenAI 的审核和 Bergeron 的检测。我们还发现困惑度过滤器对于黑盒越狱攻击完全无效，表明所有黑盒越狱提示语都高度可读。系统提示语比提示语补丁更有效，尤其对于专有模型而言。这可能是由于在模型的对齐过程中系统提示语具有更高的权重。此外，输入规范化在防御 DIA 方面被证明是无效的，表明即使只有角色标签存在，越狱仍然可能被启用。

VI. RW

A. LLM 越狱攻击

关于 LLM 越狱攻击的现有研究可以根据攻击者可获得的访问权限级别分为白盒攻击、灰盒攻击和黑盒攻击。白盒攻击涉及模型训练。GCG [27] 使用贪婪梯度搜索优化对抗性后缀。AutoDAN [73] 通过双重目标提升提示的可读性。灰盒攻击需要本地推理。一种常见的灰盒攻击是预填充攻击 [28], [29]，它在响应中植入肯定的起始内容。黑盒攻击只需要对受害者模型进行查询访问。DeepInception [14] 采用想象场景进行催眠，DRA [15] 隐藏敏感词汇。LLM-Fuzzer [74] 模仿模糊测试以生成自动化的越狱模板。上述所有攻击都集中在单轮对话上。Crescendo [19] 提出了一种多轮越狱方法，但仅限于设计用户提示。我们的 DIA 迈出了利用整个对话历史（包括用户提示、系统提示和助手响应）的第一步。

B. LLM 越狱防御

关于 LLM 越狱防御的研究通常分为基于学习和基于策略的方法。基于学习的防御通过定制化训练方法增强模型鲁棒性。例如，DRO [70] 和 DDP [57] 采用鲁棒优化生成提醒提示，而 MART [75] 引入多轮对抗训练框架以增强模型抗性。基于策略的防御则依赖操作技术，帮助模型识别并对抗恶意输入或输出。困惑度过滤器 [69] 拒绝具有高困惑度的输入。Bergeron [71] 和 SelfDefense [76] 利用第二个 LLM 监控输入或输出。RAIN [77] 采用生成方向的向前搜索，允许 LLM 自身评估其输出。尽管有这些进步，我们提出的利用历史对话操纵的攻击方法，证明了需要更全面的防御策略，这些策略应考虑多轮交互。

VII. C

在这篇论文中，我们介绍了 DIA，一种利用 LLM 聊天模板结构和精心设计的对话来利用大型语言模型漏洞的新型黑盒越狱范式。通过操纵对话历史，DIA 揭示了 LLM 安全机制中的新漏洞。我们的研究结果表明，在开发 LLM 安全措施时，必须考虑历史对话交互，强调了保护 LLM 免受对抗性利用的持续挑战。

EC

这项研究旨在突出 LLM 的安全漏洞，以加强防御对抗性滥用。所有实验均在受控环境中进行，且未将与研究无关的人员共享有害内容。人类评估研究已获得浙江大学生物医学工程与仪器科学学院伦理委员会的审查和批准。作者认为，负责任地披露这些漏洞对于推进 AI 安全和发展稳健的安全保障至关重要。

A

作者感谢匿名审稿人和编辑们提出的宝贵评论和建设性建议。

R

[1] J. Achiam 等人， ₲PT-4 技术报告，”2023 年，arXiv:2303.08774。 [2] A. Chowdhery 等人， ₱aLM：通过路径扩展语言建模，”机器学习研究杂志，第 24 卷，第 240 期，第 1-113 页，2022 年。 [3] Q. Gu， 基于 LLM 的 Go 语言编译器测试代码生成方法，”第 31 届 ACM 欧洲软件工程会议联合研讨会论文集。

软件工程，2023 年 11 月，第 2201-2203 页。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

[4] R. Koshkin, K. Sudoh, and S. Nakamura, "TransLLaMa: LLM-based simultaneous translation system," 2024, *arXiv:2402.04636*.

[5] X. Lin et al., "Data-efficient fine-tuning for LLM-based recommendation," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2024, pp. 365–374.

[6] S. Dai, C. Xu, S. Xu, L. Pang, Z. Dong, and J. Xu, "Bias and unfairness in information retrieval systems: New challenges in the LLM era," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 6437–6447.

[7] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, and W. Wang, "Pride and prejudice: LLM amplifies self-bias in self-refinement," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15474–15492.

[8] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2022, pp. 27730–27744.

[9] X. Yin, W. Lin, K. Sun, C. Wei, and Y. Chen, "A2S2-GNN: Rigging GNN-based social status by adversarial attacks in signed social networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 206–220, 2023.

[10] S. E. Trevlakis, N. Pappas, and A.-A.-A. Boulogeorgos, "Toward natively intelligent semantic communications and networking," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 1486–1503, 2024.

[11] A. Shekofteh and R. A. Chou, "SPIR with colluding and non-replicated servers from a noisy channel," in *Proc. 60th Annu. Allerton Conf. Commun., Control, Comput.*, Sep. 2024, pp. 1–6.

[12] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," 2023, *arXiv:2308.03825*.

[13] Z. Yu, X. Liu, S. Liang, Z. Cameron, C. Xiao, and N. Zhang, "Don't listen to me: Understanding and exploring jailbreak prompts of large language models," 2024, *arXiv:2403.17336*.

[14] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, "DeepInception: Hypnotize large language model to be jailbreaker," 2023, *arXiv:2311.03191*.

[15] Y. Liu, Y. Zhang, Z. Zhao, Y. Dong, G. Meng, and K. Chen, "Making them ask and answer: Jailbreaking large language models in few queries via disguise and reconstruction," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 4711–4728.

[16] M. Andriushchenko and N. Flammarion, "Does refusal training in LLMs generalize to the past tense?," 2024, *arXiv:2407.11969*.

[17] H. Li et al., "Multi-step jailbreaking privacy attacks on ChatGPT," 2023, *arXiv:2304.05197*.

[18] Z. Lin, J. Cui, X. Liao, and X. Wang, "Malla: Demystifying real-world large language model integrated malicious services," 2024, *arXiv:2401.03315*.

[19] M. Russinovich, A. Salem, and R. Eldan, "Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 2421–2440.

[20] X. Du et al., "Multi-turn jailbreaking large language models via attention shifting," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 22, pp. 23814–23822.

[21] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[22] G. Team et al., "Gemma 2: Improving open language models at a practical size," 2024, *arXiv:2408.00118*.

[23] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53728–53741.

[24] J. Hong, N. Lee, and J. Thorne, "ORPO: Monolithic preference optimization without reference model," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 11170–11189.

[25] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.

[26] (2024). *Ollama: A Toolkit for Large Language Models*. [Online]. Available: https://github.com/ollama/ollama

[27] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Zico Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, *arXiv:2307.15043*.

[28] M. Andriushchenko, F. Croce, and N. Flammarion, "Jailbreaking leading safety-aligned LLMs with simple adaptive attacks," 2024, *arXiv:2404.02151*.

[29] L. Lv et al., "AdaPPA: Adaptive position pre-fill jailbreak attack approach targeting LLMs," 2024, *arXiv:2409.07503*.

[30] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022, *arXiv:2211.09527*.

[31] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "YaRN: Efficient context window extension of large language models," 2023, *arXiv:2309.00071*.

[32] W. Xiong et al., "Effective long-context scaling of foundation models," 2023, *arXiv:2309.16039*.

[33] H. Tang et al., "RazorAttention: Efficient KV cache compression through retrieval heads," 2024, *arXiv:2407.15891*.

[34] J. Baek, A. Hussain, D. Liu, N. Vincent, and L. H. Kim, "Open WebUI: An open, extensible, and usable interface for AI interaction," 2025, *arXiv:2510.02546*.

[35] (2025). *Lobechat: An Open-Source, Modern Design CHATGPT/LLMS UI/framework*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/lobehub/lobe-chat

[36] Oobabooga. (2025). *Text Generation WEBUI: The Definitive Web UI For Local AI, With Powerful Features and Easy Setup* Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/oobabooga/text-generation-webui

[37] (2025). *Streamlit: A Faster Way To Build and Share Data Apps*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/streamlit/streamlit

[38] (2025). *Gradio: Build and Share Delightful Machine Learning Apps, All in Python*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/gradio-app/gradio

[39] (2025). *Jan is an Open Source Alternative To Chatgpt That Runs 100% Offline on Your Computer*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/janhq/jan

[40] (2025). *Chatbox: User-Friendly Desktop Client App for Ai Models/llms*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/chatboxai/chatbox

[41] Danny-avila. (2025). *Librechat*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/danny-avila/LibreChat

[42] SillyTavern. (2025). *Sillytavern: Llm Frontend for Power Users*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/SillyTavern/SillyTavern

[43] H. Face. (2025). *Chat-UI: Open Source Codebase Powering the Huggingchat App*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/huggingface/chat-ui

[44] (2025). *Bionic-GPT: Bionic is an On-Premise Replacement for Chatgpt, Offering the Advantages of Generative Ai While Maintaining Strict Data Confidentiality*. Accessed: Nov. 10, 2025. [Online]. Available: https://github.com/bionic-gpt/bionic-gpt

[45] X. Qi et al., "Safety alignment should be made more than just a few tokens deep," 2024, *arXiv:2406.05946*.

[46] N. Rajani, L. Tunstall, E. Beeching, N. Lambert, A. M. Rush, and T. Wolf. (2023). *No Robots*. [Online]. Available: https://huggingface.co/datasets/HuggingFaceH4/norobots

[47] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, "Length-controlled AlpacaEval: A simple way to debias automatic evaluators," 2024, *arXiv:2404.04475*.

[48] Hugging Face. (2025). *Open LLM Leaderboard*. Accessed: Nov. 8, 2025. [Online]. Available: https://huggingface.co/spaces/open-llm-leaderboard/openllmleaderboard

[49] Z. Zhang et al., "H2O: heavy-hitter Oracle for efficient generative inference of large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 34661–34710.

[50] P. Chao et al., "JailbreakBench: An open robustness benchmark for jailbreaking large language models," 2024, *arXiv:2404.01318*.

[51] S. Bird, E. Klein, and E. Loper, *Natural Language Processing With Python: Analyzing Text With the Natural Language Toolkit*. Sebastopol, CA, USA: O'Reilly, 2009.

[52] B. Cao, Y. Cao, L. Lin, and J. Chen, "Defending against alignment-breaking attacks via robustly aligned LLM," 2023, *arXiv:2309.14348*.

[53] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[54] X. Qi et al., "Fine-tuning aligned language models compromises safety, even when users do not intend to!," 2023, *arXiv:2310.03693*.

[55] P. Ding et al., "A wolf in Sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily," 2023, *arXiv:2311.08268*.

[56] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Apr. 2023, pp. 32–42.

[57] C. Xiong, X. Qi, P.-Y. Chen, and T.-Y. Ho, "Defensive prompt patch: A robust and interpretable defense of LLMs against jailbreak attacks," 2024, *arXiv:2405.20099*.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

[4] R. Koshkin, K. Sudoh 和 S. Nakamura, “TransLLaMa: 基于 LLM 的实时翻译系统,” 2024, arXiv:2402.04636. [5] X. Lin 等人, “面向 LLM 推荐的数据高效微调,” 在第 47 届国际 ACM SIGIR 研究与发展会议论文集中.

Inf. Retr., 2024 年 7 月, 第 365-374 页。

[6] 戴思，徐超，徐思，庞立，董哲，徐静， 偏差与不公平
在信息检索系统中：LLM 时代的新挑战，”在 30 届 ACM SIGKDD 知识发现与数据挖掘会议论文集中，8 月。
2024, 第 6437–6447 页。

[7] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, 和 W. Wang, “Pride and
偏见：LLM 在自我完善中放大自我偏见，”在 2024 年计算机语言学协会第 62 届年会上，第 15474-15492 页。

[8] L. Ouyang 等人， 训练语言模型以遵循指令”
“人类反馈”，在《先进神经信息处理系统会议论文集》（NIPS）2022 年会议论文中，第 27730-27744 页。

[9] X. Yin, W. Lin, K. Sun, C. Wei, 和 Y. Chen, “A2S2-GNN: Rig-
ging 基于 GNN 的社会地位在签名社交网络中的对抗性攻击，”IEEE 信息技术与安全，第 18 卷，第 206-220 页，2023 年。

[10] S. E. Trevlakis, N. Pappas, 和 A.-A.-A. Boulogeorgos, “迈向”
natively intelligent semantic communications and networking,” IEEE Open J. Commun. Soc., vol. 5, pp. 1486–1503, 2024.

[11] A. Shekofteh and R. A. Chou, “SPIR with colluding and non-replicated servers from a noisy channel,” in Proc. 60th Annu. Allerton Conf.
Commun., Control, Comput., Sep. 2024, pp. 1–6.

[12] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, “ Do anything
now’：对自然场景下大型语言模型中的越狱提示进行表征和评估，”2023 年，arXiv:2308.03825。

[13] Z. Yu, X. Liu, S. Liang, Z. Cameron, C. Xiao, 和 N. Zhang, “不要
听我说： 理解和探索大型语言模型的越狱提示”，2024 年，arXiv:2403.17336。

[14] 李翔, 周志, 朱俊, 姚杰, 刘涛, 和 韩冰, “DeepInception: 使大型语言模型成为越狱者,” 2023, arXiv:2311.03191.

[15] 刘勇，张勇，赵哲，董勇，孟刚，陈凯， 使

他们提问和回答： 通过伪装和重建在少量查询中破解大型语言模型”，在《第 33 届 USENIX 安全研讨会论文集》，2024 年，第 4711-4728 页。

[16] M. Andriushchenko 和 N. Flammarion, “LLMs 的拒绝训练是否泛化到过去时态？,” 2024, arXiv:2407.11969. [17] H. Li 等人, “针对 ChatGPT 的多步越狱隐私攻击,” 2023, arXiv:2304.05197.

[18] Z. Lin, J. Cui, X. Liao, 和 X. Wang, “Malla: 揭开真实-

“在大型语言模型中集成恶意服务”，2024 年，arXiv:2401.03315.

[19] M. Russinovich、A. Salem 和 R. Eldan， 太好了，现在写一篇关于那个的文章：The crescendo 多轮 LLM 越狱攻击”，在《会议论文集》中。
第 34 届 USENIX 安全研讨会（USENIX Secur.），2024 年，第 2421–2440 页。

[20] 杜雪等， 多轮破解大型语言模型通过注意力”
"迁移"，在 AAAI 人工智能会议论文集中，2025 年，第 39 卷，第 22 期，第 23814-23822 页。

[21] H. Touvron 等人， LLaMA：开放且高效的基础语言模型”，2023，arXiv:2302.13971。 [22] G. Team 等人， Gemma 2：在实用规模上改进开放语言模型”，2024，arXiv:2408.00118。 [23] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning 和

C. Finn, “直接偏好优化：你的语言模型本质上是一个奖励模型,” 在《先进神经信息处理系统会议论文集》2023, 第 53728–53741 页。

[24] J. Hong, N. Lee, 和 J. Thorne, “ORPO: Monolithic preference opti-
“无需参考模型的优化”，在《自然语言处理经验方法会议论文集》2024 年，第 11170–11189 页。

[25] W. Kwon 等人， 高效的大型语言
使用 PagedAttention 的模型服务，”在 2023 年 10 月第 29 届操作系统原理研讨会论文集中，第 611-626 页。

[26] (2024). Ollama: 用于大型语言模型的工具包。[在线]. 可用-
able: https://github.com/ollama/ollama [27] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Zico Kolter, and M. Fredrikson,
“面向对齐语言模型的通用和可迁移对抗攻击,” 2023, arXiv:2307.15043.

[28] M. 安德鲁申科, F. 克罗切, 和 N. 弗拉马里翁, “越狱
在“针对安全对齐 LLMs 的简单自适应攻击”，2024 年，arXiv:2404.02151。

[29] L. Lv 等人， AdaPPA：针对 LLMs 的自适应位置预填充越狱攻击方法，”2024 年，arXiv:2409.07503。 [30] F. Perez 和 I. Ribeiro， 忽略之前的提示：语言模型的攻击技术，”2022 年，arXiv:2211.09527。

[31] 彭博，奎斯奈尔・J，范・H，和希波勒・E， YaRN：大型语言模型的效率上下文窗口扩展”，2023，arXiv:2309.00071。 [32] 熊伟等， 基础模型的有效长上下文扩展”.

2023, arXiv:2309.16039.

[33] H. Tang 等人， RazorAttention：通过检索头实现高效的 KV 缓存压缩”，2024 年，arXiv:2407.15891。 [34] J. Baek, A. Hussain, D. Liu, N. Vincent 和 L. H. Kim，“Open WebUI：

一个开放、可扩展且可用的 AI 交互界面，” 2025, arXiv:2510.02546.

[35] (2025). Lobechat：一个开源的、现代设计的 CHATGPT/LLMs
UI/框架。访问时间：2025 年 11 月 10 日。[在线]. 获取方式：https://github.com/lobehub/lobe-chat

[36] Oobabooga. (2025). 文本生成 WEBUI: 终极 Web 界面
本地 AI，功能强大且易于设置，访问时间：2025 年 11 月 10 日。[在线]. 可获得：https://github.com/oobabooga/text-generationwebui [37] (2025). Streamlit：一种更快构建和共享数据应用的方法。

访问时间：2025 年 11 月 10 日。[在线]. 获取方式：https://github.com/streamlit/streamlit

[38] (2025). Gradio：构建和分享令人愉悦的机器学习应用，
全部使用 Python 编写。访问时间：2025 年 11 月 10 日。[在线]. 获取方式：https://github.com/gradio-app/gradio

[39] (2025). Jan 是一个开源的 ChatGPT 替代品，可 100%运行
离线，在你的电脑上。访问时间：2025 年 11 月 10 日。[在线]. 可用：https://github.com/janhq/jan [40] (2025)。聊天框：用户友好的桌面客户端应用程序，用于 AI 模型/LLMs。

访问时间：2025 年 11 月 10 日。[在线]. 可获得：https://github.com/chatboxai/chatbox [41] Danny-avila. (2025). Librechat 访问时间：2025 年 11 月 10 日。[在线]
可用：https://github.com/danny-avila/LibreChat

[42] SillyTavern. (2025). Sillytavern: LLM 前端，为高级用户设计。
访问时间：2025 年 11 月 10 日。 [在线]. 可获得：https://github.com/SillyTavern/SillyTavern

[43] H. Face. (2025). Chat-UI：为 Huggingchat 应用提供开源代码库。
访问时间：2025 年 11 月 10 日。 [在线]. 可获得：https://github.com/huggingface/chat-ui

[44] (2025). Bionic-GPT：Bionic 是 ChatGPT 的本地替代品，
提供生成式 AI 的优势同时保持严格的数据保密性。访问时间：2025 年 11 月 10 日。[在线]. 可获得：https://github.com/bionic-gpt/bionic-gpt

[45] X. Qi 等人， 安全对齐应该不仅仅深入几个 token，” 2024，arXiv:2406.05946。 [46] N. Rajani, L. Tunstall, E. Beeching, N. Lambert, A. M. Rush 和

T. Wolf. (2023). No Robots. [在线]. 可获得：https://huggingface.co/datasets/HuggingFaceH4/norobots

[47] Y. Dubois, B. Galambosi, P. Liang 和 T. B. Hashimoto, “长度控制的 AlpacaEval：一种简单的方法来减少自动评估器的偏差,”
2024, arXiv:2404.04475.

[48] Hugging Face. (2025). Open LLM 排行榜. 访问时间：2025 年 11 月 8 日.
[在线]. 可获得：https://huggingface.co/spaces/open-llm-leaderboard/openllmleaderboard

[49] 张智等， R2O：高效生成推理的大型语言模型的强力预言机”，在《先进神经信息处理会议论文集》中
Syst., 卷 36, 2024 年, 第 34661–34710 页。

[50] P. Chao 等人， JailbreakBench: 一个用于破解大型语言模型的开放鲁棒性基准，”2024 年，arXiv:2404.01318。 [51] S. Bird, E. Klein 和 E. Loper， 自然语言处理》

Python: 使用自然语言工具包分析文本。美国加利福尼亚州塞巴斯蒂奥普：奥莱利出版社，2009 年。

[52] 曹博，曹宇，林立，和陈杰， 通过鲁棒对齐的 LLM 防御对齐破坏攻击”，2023，arXiv:2309.14348。 [53] N. Reimers 和 I. Gurevych， Sentence-BERT: Sentence embeddings

使用 Siamese BERT 网络，”在第九届国际自然语言处理联合会议暨自然语言处理经验方法会议论文集中
(EMNLP-IJCNLP), 2019, pp. 3980–3990.

[54] X. Qi 等人， 微调对齐语言模型会危及安全，即使用户无意中！”，2023，arXiv:2310.03693。 [55] P. Ding 等人， 披着羊皮的狼：通用嵌套”

越狱提示可以轻易欺骗大型语言模型，” 2023, arXiv:2311.08268.

[56] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, 和 E. Wong,
“在二十次查询中越狱黑盒大型语言模型，” 在 IEEE 安全可信机器学习会议 (SaTML) 会议记录中，2023 年 4 月，第 32-42 页。

[57] C. Xiong, X. Qi, P.-Y. Chen, 和 T.-Y. Ho, “防御提示补丁：对越狱攻击的 LLMs 的鲁棒和可解释防御,”
2024, arXiv:2405.20099.

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。

[58] G. Deng et al., "MASTERKEY: Automated jailbreaking of large language model chatbots," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–16.
[59] Z. Chang, M. Li, Y. Liu, J. Wang, Q. Wang, and Y. Liu, "Play guessing game with LLM: Indirect jailbreak attack with implicit clues," 2024, *arXiv:2402.09091*.
[60] Y. Zeng, Y. Wu, X. Zhang, H. Wang, and Q. Wu, "AutoDefense: Multi-agent LLM defense against jailbreak attacks," 2024, *arXiv:2403.04783*.
[61] L. Team. (2024). *Meta Llama Guard 2*. [Online]. Available: https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard2/MODELCARD.md
[62] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
[63] Y. Zhang, L. Ding, L. Zhang, and D. Tao, "Intention analysis makes LLMs a good jailbreak defender," in *Proc. 31st Int. Conf. Comput. Linguistics*, 2024, pp. 2947–2968.
[64] C. Barrett et al., "Identifying and mitigating the security risks of generative AI," *Found. Trends Privacy Secur.*, vol. 6, no. 1, pp. 1–52, 2023.
[65] Y. Fu, Y. Li, W. Xiao, C. Liu, and Y. Dong, "Safety alignment in NLP tasks: Weakly aligned summarization as an in-context attack," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, Aug. 2024, pp. 8483–8502. [Online]. Available: https://aclanthology.org/2024.acl-long.461
[66] J. Ji et al., "BeaverTails: Towards improved safety alignment of LLM via a human-preference dataset," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 24678–24704.
[67] AI@Meta. (2024). *Llama 3 Model Card*. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODELCARD.md
[68] (2024). *Openai Moderation Api*. [Online]. Available: https://platform.openai.com/docs/guides/moderation
[69] N. Jain et al., "Baseline defenses for adversarial attacks against aligned language models," 2023, *arXiv:2309.00614*.
[70] C. Zheng et al., "On prompt-driven safeguarding for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 61593–61613.
[71] M. Pisano et al., "Bergeron: Combating adversarial attacks through a conscience-based alignment framework," 2023, *arXiv:2312.00029*.
[72] Z. Xu, Y. Liu, G. Deng, Y. Li, and S. Picek, "A comprehensive study of jailbreak attack versus defense for large language models," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 7432–7449.
[73] S. Zhu et al., "AutoDAN: Interpretable gradient-based adversarial attacks on large language models," 2023, *arXiv:2310.15140*.
[74] J. Yu, X. Lin, Z. Yu, and X. Xing, "LLM-Fuzzer: Scaling assessment of large language model jailbreaks," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 4657–4674.
[75] S. Ge et al., "MART: Improving LLM safety with multi-round automatic red-teaming," 2023, *arXiv:2311.07689*.
[76] M. Phute et al., "LLM self defense: By self examination, LLMs know they are being tricked," 2023, *arXiv:2308.07308*.
[77] Y. Li, F. Wei, J. Zhao, C. Zhang, and H. Zhang, "RAIN: Your language models can align themselves without finetuning," 2023, *arXiv:2309.07124*.



Wenlong Meng received the B.S. degree in electronic information and communication from the Huazhong University of Science and Technology, Wuhan, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Zhejiang University, Hangzhou, China. His current research interests include trustworthy and privacy-preserving LLM applications. He has published papers on top-tier security conferences, such as USENIX Security, NDSS, and ACM CCS. Particularly, one of his research has been honored with the Distinguished Paper Award at NDSS 2024.



Fan Zhang is currently pursuing the bachelor's degree with Zhejiang University, Haining International Campus, China. He has been recommended for postgraduate admission to Zhejiang University and will pursue graduate studies there. His research interests include federated learning and the security of large language models (LLMs), with a focus on robust and privacy-preserving learning and trustworthy model deployment.



Wendao Yao is currently pursuing the dual B.Eng. degree in computer engineering with the ZJU-UIUC Institute, Zhejiang University, Haining, China, as part of a dual-degree program jointly offered by Zhejiang University and the University of Illinois Urbana-Champaign. His research interests include machine learning security, privacy-preserving natural language processing, and trustworthy large language models.



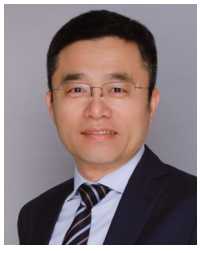
Zhenyuan Guo received the B.E. degree from Zhengzhou University and the M.E. degree from Beijing Institute of Technology. He is currently pursuing the Ph.D. degree with Zhejiang University. His research interests include large language models, long-context reasoning, and model security.



Yuwei Li received the Ph.D. degree in computer science and technology from Zhejiang University, Hangzhou, China, in 2021. She is an Associate Professor with the College of Electronic Engineering, National University of Defense Technology. Her research interests include system and software security, fuzz testing, and vulnerability detection.



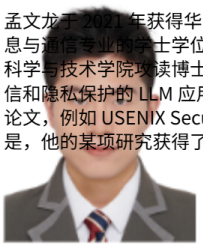
Chengkun Wei received the Ph.D. degree in computer science from Zhejiang University. He is a ZJU 100-Young Professor with the College of Computer Science and Technology, Zhejiang University. He has published over ten papers in prestigious international journals and conferences, including CCS, NDSS, SIGCOMM, ATC, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, and IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY. His research interests include data security (differential privacy and AI security) and architecture design for cloud computing (network vSwitch and trusted execution environment (TEE) operating system design). He was honored with the NDSS 2024 Distinguished Paper Award. He is a reviewer of several international journals.



Wenzhi Chen (Member, IEEE) received the Ph.D. degree from the College of Computer Science and Technology, Zhejiang University. He is a Professor with the College of Computer Science and Technology, Zhejiang University; and the Director of the Information Technology Center, Zhejiang University. He is the Vice Dean of the College of Computer Science and Technology, Zhejiang University. His current research interests include embedded systems and their applications, computer architecture, computer system software, and information security. He is a member of ACM and the ACM Education Council. He was a recipient of the NDSS 2024 Distinguished Paper Award.

Authorized licensed use limited to: Hainan University. Downloaded on April 15,2026 at 08:40:02 UTC from IEEE Xplore. Restrictions apply.

[58] G.邓等， “MASTERKEY：自动化破解大型语言模型”
在《网络与分布式系统安全研讨会论文集》2024 年，第 1-16 页中， “语言模型聊天机器人，”
[59] 张智，李明，刘勇，王静，王强，刘勇， “玩猜谜
使用 LLM 的游戏：带有隐式线索的间接越狱攻击，”2024 年，arXiv:2402.09091。
[60] 曾勇，吴勇，张翔，王浩，吴强， “AutoDefense：多智能体 LLM 防御对抗越狱攻击，”2024 年，arXiv:2403.04783。 [61] L.团队。 “2024 年）。Meta Llama Guard 2。[在线]。可访问：
https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard2/MODELCARD.md [62] A. Grattafiori 等人， “The llama 3 herd of models,” 2024, arXiv:2407.21783.
[63] 张勇，丁磊，张丽，和陶东， “意图分析使 LLMs 成为良好的越狱防御者，”在第 31 届国际计算机会议论文集中
语言学，2024 年，第 2947-2968 页。
[64] C. Barrett 等人， “识别和缓解安全风险”
生成式 AI，”隐私与安全趋势，第 6 卷，第 1 期，第 1-52 页，2023 年。
[65] Y. Fu, Y. Li, W. Xiao, C. Liu, 和 Y. Dong, “NLP 任务中的安全对齐：作为情境攻击的弱对齐摘要，”
在《第 62 届计算语言学协会年度会议论文集》，2024 年 8 月，第 8483–8502 页。[在线]. 可获得：https://aclanthology.org/2024.acllong.461
[66] J. Ji 等人， “BeaverTails：迈向改进的 LLM 安全对齐”
通过一个人类偏好数据集，”在《先进神经信息处理系统会议论文集》2023 年，第 24678–24704 页。
[67] AI@Meta. (2024). Llama 3 模型说明卡。[在线]. 获取方式：https://github.com/meta-llama/llama3/blob/main/MODELCARD.md [68] (2024).
platform.openai.com/docs/guides/moderation
[69] N. Jain 等人， “针对对齐语言模型的对抗攻击的基线防御，”2023 年，arXiv:2309.00614。 [70] C. Zheng 等人， “关于大型语言模型的提示驱动保护，”在 2024 年国际机器学习会议论文集中，第 61593-61613 页。 [71] M. Pisano 等人，“Bergeron：通过基于良知的对齐框架对抗对抗攻击，”2023 年，arXiv:2312.00029。 [72] Z. Xu, Y. Liu, G. Deng, Y. Li 和 S. Picek， “对大型语言模型的越狱攻击与防御的全面研究，”在论文集中。
在计算语言学协会期刊 ACL，2024 年，第 7432-7449 页。
[73] S. Zhu 等人， “AutoDAN：针对大型语言模型的基于梯度的可解释对抗攻击”，2023，arXiv:2310.15140。 [74] J. Yu, X. Lin, Z. Yu 和 X. Xing， “LLM-Fuzzer：规模评估”
大型语言模型越狱，”在 2024 年第 33 届 USENIX 安全研讨会上，第 4657-4674 页。
[75] S. Ge 等人， “MART：通过多轮自动红队提升 LLM 安全性”，2023，arXiv:2311.07689。 [76] M. Phute 等人， “LLM 自我防御：通过自我审视，LLMs 知道它们正被欺骗”，2023，arXiv:2308.07308。 [77] Y. Li, F. Wei, J. Zhao, C. Zhang 和 H. Zhang， “RAIN：您的”
语言模型可以在不进行微调的情况下自我对齐，”2023 年，arXiv:2309.07124。



孟文龙于 2021 年获得华中科技大学（中国武汉）电子信息与通信专业的学士学位。目前，他正在浙江大学计算机科学与技术学院攻读博士学位。他的当前研究方向包括可信和隐私保护的 LLM 应用。他在顶级安全会议上发表过论文，例如 USENIX Security、NDSS 和 ACM CCS。特别是，他的某项研究获得了 NDSS 2024 的杰出论文奖。



张帆目前就读于中国浙江大学海宁国际校区，攻读学士学位。他已被推荐进入浙江大学攻读研究生，并将在那里继续深造。他的研究兴趣包括联邦学习和大型语言模型（LLMs）的安全，重点关注鲁棒和隐私保护的学习以及可信模型部署。



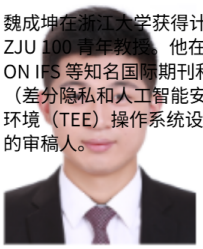
姚文道目前就读于浙江大学 ZJU-UIUC 学院，攻读计算机工程双学士学位，作为浙江大学与伊利诺伊大学厄巴纳-香槟分校联合提供的双学位项目的一部分。他的研究兴趣包括机器学习安全、隐私保护的自然语言处理以及可信的大型语言模型。



郭振元获得郑州大学的工学学士学位和北京理工大学的工学硕士学位。他目前就读于浙江大学，攻读博士学位。他的研究兴趣包括大型语言模型、长上下文推理和模型安全。



余伟在 2021 年获得浙江大学计算机科学与技术博士学位。她是国防科技大学电子工程学院副教授。她的研究兴趣包括系统与软件安全、模糊测试和漏洞检测。



魏成坤在浙江大学获得计算机科学博士学位。他是浙江大学计算机科学与技术学院 ZJU 100 青年教授。他在 CCS、NDSS、SIGCOMM、ATC、IEEE TD SC 和 IEEE T ON IFS 等知名国际期刊和会议上发表了十多篇论文。他的研究兴趣包括数据安全（差分隐私和人工智能安全）以及云计算的架构设计（网络虚拟交换机和可信执行环境（TEE）操作系统设计）。他获得了 NDSS 2024 杰出论文奖。他是多个国际期刊的审稿人。



陈文志（IEEE 会员）获得浙江大学计算机科学与技术学院的博士学位。他是浙江大学计算机科学与技术学院的教授，同时也是浙江大学信息技术中心主任。他是浙江大学计算机科学与技术学院的副院长。他目前的研究兴趣包括嵌入式系统及其应用、计算机体系结构、计算机系统软件和信息安全。他是 ACM 会员和 ACM 教育委员会成员。他曾获得 NDSS 2024 杰出论文奖。

授权许可使用仅限于：海南大学。于 2026 年 4 月 15 日 08:40:02 UTC 从 IEEE Xplore 下载。适用限制。